The Densest Co-Activated Group Search Problem

Tianyou Gao
Kyoto University
Kyoto, Japan
gao.tianyou.66z@st.kyoto-u.ac.jp

Keishi Tajima
Kyoto University
Kyoto, Japan
tajima@i.kyoto-u.ac.jp

Takayuki Ito
Kyoto University
Kyoto, Japan
ito@i.kyoto-u.ac.jp

***Abstract*—Many viral marketing campaigns promote products designed for joint use by multiple users, e.g., multiplayer games and collaborative work management software. In such campaigns, the classical influence measure in influence maximization captures overall diffusion reach but not joint activation, and therefore cannot fully characterize campaign effectiveness. To capture this aspect, we formulate the *Densest Co-Activated Group Search (DCGS)* problem, which seeks a user group with maximum co-activation density under diffusion from a given seed set. DCGS is computationally challenging because the underlying co-activation structure is implicit and quadratic in size, making it prohibitively expensive to materialize on large graphs. To address this, we propose a two-stage algorithm. The first stage screens promising user pairs, while the second performs certified sampling integrated with densest-subgraph optimization. Using weighted peeling or a min-cut-based densest-subgraph solver for the optimization component yields two algorithmic variants, which achieve $(2+\varepsilon)$- and $(1+\varepsilon)$-approximation guarantees with high probability, respectively. Experiments on eleven real-world datasets show that both algorithmic variants achieve co-activation densities comparable to those of strong heuristic baselines while substantially reducing memory consumption.**

***Index Terms*—influence diffusion, co-activation similarity, densest subgraph, approximation algorithm**

I. Introduction

Diffusion phenomena are ubiquitous in social systems, where information, opinions, behaviors, and innovations spread from person to person through networked interactions [1]–[6]. Based on such phenomena, a broad line of research has studied computational problems on online social networks (OSNs), e.g., estimating the influence of given seed sets [4], [7], [8], selecting a set of seed users to maximize influence spread [4], [9], [10], and designing interventions to limit undesirable diffusion [11]–[13]. However, the objectives considered in these studies do not directly address the needs arising in the following group-oriented applications.

- **Multiplayer game promotion.** Consider a gaming platform promoting a new multiplayer game on OSNs. Major gaming platforms, e.g., Xbox[1] and PlayStation[2], support online multiplayer games and social interactions among players. Before launching the campaign, the platform may engage a fixed set of sponsored influencers as seed users, who then promote the game through their social interactions. After launching the campaign, the platform may wish to identify a group of users who are likely to adopt the game together, since multiplayer games derive much of their value from the presence of multiple active users. The discovered group can then be targeted through a group-oriented follow-up campaign.
- **Collaborative software deployment.** Consider a company deploying a collaborative work management product, e.g., Asana[3] or Microsoft Teams[4], through its internal communication network. The company may first provide the product to a fixed set of internal advocates as seed users, who then encourage other employees to adopt it through workplace interactions. After the initial deployment, the company may wish to identify a group of employees who are likely to adopt the product together. Identifying such a group is important because collaborative software derives much of its value from multiple users participating in the same shared workflow. The discovered group can then be targeted through a joint onboarding program.

In both applications, the practical goal is to identify users likely to adopt the product together so that they can be targeted through follow-up actions, e.g., group-oriented promotion and joint onboarding. For multiplayer games, such actions may encourage users to continue playing together, while for collaborative software, they may encourage entire teams to adopt the same tool and avoid fragmented use. For two users u and v, let A_u and A_v denote their activation events. At the pairwise level, their tendency to adopt together is captured by the joint activation probability $\Pr[A_u \cap A_v]$, rather than only by the individual activation probabilities $\Pr[A_u]$ and $\Pr[A_v]$. For example, suppose that $\Pr[A_u] = \Pr[A_v] = 0.6$ and that their activation events are independent. Their joint activation probability is then only $\Pr[A_u \cap A_v] = 0.36$. For a multiplayer game, this means that u and v may each be likely to adopt the game while remaining substantially less likely to adopt it together, thereby limiting the group-oriented value of the campaign. More generally, pairwise co-activation naturally aggregates to the group level, since for a user group U, the sum of pairwise co-activation probabilities equals the expected number of jointly activated user pairs within U. By contrast, most existing diffusion studies focus on the standard measure of *influence*, i.e., the expected number of activated users,

This work was supported by JSPS KAKENHI Grant Number JP22H00533.

[1]https://www.xbox.com/en-us

[2]https://www.playstation.com/en-us

[3]https://asana.com

[4]https://support.microsoft.com/en-us/teams

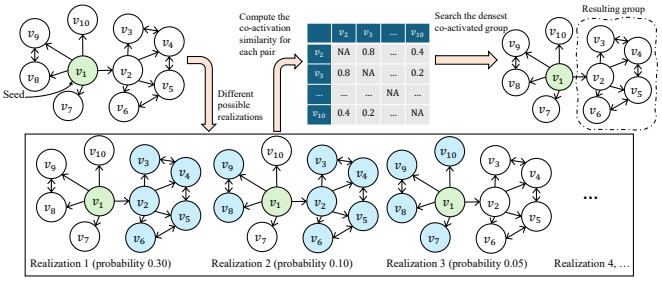


Fig. 1: An illustration of the DCGS problem. Starting from the seed set $\{v_1\}$ (the green node), different diffusion realizations activate different subsets of nodes. For example, realization 1 occurs with probability 0.3 and activates nodes $v_2, \ldots, v_6$ (the blue nodes). The distribution over diffusion realizations induces the pairwise co-activation similarities shown in the table, where, e.g., v_2 and v_3 are jointly activated with probability 0.8. DCGS searches for a group of non-seed nodes with high co-activation density under the densest-subgraph objective. In this example, the resulting group is $\{v_2, \ldots, v_5\}$. The realization probabilities are shown only for illustration. Our algorithms sample realizations from the diffusion model without enumerating all possible realizations.

equivalently, the sum of individual activation probabilities [4]. Although this measure is natural for evaluating the overall extent of diffusion, it does not capture the joint activation structure required by the applications above.

To capture the group-oriented objective, we introduce a pairwise measure called *co-activation similarity*, defined as the probability that two users are jointly activated when diffusion starts from a given seed set. Based on this measure, we define a new optimization problem called the *Densest Co-activated Group Search (DCGS)* problem. In simple terms, DCGS seeks a group of non-seed users with high internal *co-activation density*. Notably, our *density* measure follows the standard densest-subgraph objective [14], specialized to the co-activation setting. Specifically, it is defined as the total co-activation similarity of all user pairs within the group with similarity at least a prescribed threshold τ, divided by the group size. Figure 1 illustrates how the DCGS problem is defined.

Modeling choices. We restrict the output group to non-seed users because the seed users are externally specified and activated at the beginning of diffusion, whereas our goal is to identify downstream users who are likely to be activated together. Including seed users could also bias the solution toward them because their activation is deterministic. The threshold τ retains sufficiently strong co-activation relationships while inducing a sparse similarity graph that supports scalable computation. The experimental results reported in Section VII-B4 further show that decreasing τ yields only modest improvements in the objective value.

Challenges. As in many optimization problems, the main difficulty is not merely to find a good empirical solution, but to find one with good theoretical guarantees. Overcoming this difficulty, however, requires confronting the substantial computational challenges inherent in DCGS, which go beyond those of standard influence estimation. The thresholded co-activation structure is implicit, and computing the co-activation similarity of even a fixed user pair exactly is #P-hard [15]. Standard Monte Carlo sampling, a common technique for influence estimation [4], is costly even for estimating a single aggregate influence value and becomes prohibitive when simultaneously estimating many pairwise co-activation probabilities. Reverse influence sampling [16], a state-of-the-art technique for influence maximization, is also not directly applicable because each reverse-reachable sample targets a single node, whereas DCGS requires joint activation information for many node pairs.

To address these challenges, we develop a two-stage algorithm. The first stage uses pilot samples to screen user pairs that are likely to exceed the co-activation threshold, thereby reducing the pair search space before accurate estimation is performed. The second stage progressively refines the retained pairwise estimates and interleaves sampling with densest-subgraph optimization at a sequence of checkpoints. At each checkpoint, the algorithm solves a weighted densest-subgraph subproblem and uses simultaneous confidence bounds to certify the current solution, allowing early termination. Using weighted peeling [17] or a min-cut-based solver [18] for this subproblem yields two variants with $(2+\varepsilon)$- and $(1+\varepsilon)$-approximation guarantees with high probability, respectively.

Technically, designing the pilot sampling scheme in the first stage is nontrivial because it must retain only a small subset of candidate pairs while preserving, with high probability, all pairs needed for the subsequent approximation guarantee. Adapting progressive sampling with certification (a technique widely used in influence maximization [19]–[23]) in the second stage is also nontrivial because it must handle pairwise co-activation estimates while certifying the resulting densest-subgraph solution.

We conduct extensive experiments on eleven real-world datasets to evaluate the proposed algorithms. The results show that our algorithms achieve co-activation densities comparable to those obtained by strong heuristic baselines. Meanwhile, our algorithms substantially reduce memory consumption and remain applicable to large-scale graphs and large seed sets for which the baselines become impractical. These empirical advantages are achieved while retaining certified sampling accuracy and provable approximation guarantees. Although the proposed algorithms require more runtime than heuristic baselines with fixed sampling budgets due to statistical certification, they remain applicable to all datasets where a direct comparison is feasible. Overall, our methods match the solution quality of strong heuristics while overcoming their memory bottleneck and providing rigorous theoretical guarantees.

Our main contributions are summarized as follows.

(1) We formulate the DCGS problem for discovering densely co-activated user groups under diffusion.

(2) We develop a two-stage algorithm for DCGS, with both

a peeling-based implementation and an exact min-cut-based implementation.

(3) We establish $(2+\varepsilon)$- and $(1+\varepsilon)$-approximation guarantees for the peeling-based and min-cut-based algorithms, respectively.

(4) We conduct extensive experiments on real-world datasets, which demonstrate the effectiveness and scalability of the proposed methods.

Note. The full version of this paper, including an appendix containing all proofs and additional experimental results, is available at https://anonymous.4open.science/r/dcgs-VLDB/appendix.pdf.

II. Related Work

Our problem lies at the intersection of influence diffusion, group discovery, and densest subgraph search.

a) Influence diffusion: Influence diffusion, particularly influence maximization, is a well-studied topic in social network analysis [1], [4], [9], [24]. A large body of prior work focuses on node-level diffusion, e.g., topic-aware influence diffusion [25]–[28], time-aware influence diffusion [29]–[31], and location-aware influence diffusion [32]–[34].

Recently, some studies have considered selecting seed sets to maximize group-level objectives derived from the activated node sets produced by diffusion realizations [35]–[38]. Another related direction is fair influence maximization, which promotes an equitable distribution of activation outcomes across groups over diffusion realizations [39]–[43]. Related directions also include cascade stability across diffusion realizations [44], interaction activity maximization [45], and triangular stability maximization within diffusion outcomes [46]. Although the above studies optimize various group-related objectives defined over diffusion outcomes, they focus on seed selection, whereas DCGS takes the seed set as given and searches for a co-activated group.

b) Group discovery: A broad line of research focuses on group discovery based on structural patterns [47]–[49], attribute information [50], or higher-order structures in graphs [51]. In contrast, influential community search seeks communities that are structurally cohesive and influential under predefined node-level influence measures [52]–[57]. Another line of work studies diffusion-aware community partitioning, which groups nodes according to their mutual influence under diffusion [58]–[60]. The study most closely related to DCGS is that of Chang et al. [61], which seeks the subgraph with the highest influenced expectations among those satisfying a prescribed structural constraint in the original network, e.g., a D-core [62]. DCGS instead seeks a group of users that is densest with respect to their pairwise co-activation similarities induced by stochastic diffusion. Section VII-B6 empirically compares the groups identified by the two formulations.

c) Densest subgraph search: Classical studies have established both exact and approximate algorithmic foundations for the densest subgraph search problem [14], [17], [63], [64]. Beyond the classical setting, many variants of densest subgraph search have also been studied [65]–[71]. Related studies have also considered densest subgraph search in uncertain or stochastic graph settings [72]–[76]. However, these studies do not explicitly consider influence diffusion as the source of uncertainty, whereas in our problem the target structure is induced by stochastic diffusion and reflects co-activation patterns among nodes.

III. Problem Formulation

This section introduces the diffusion model, the co-activation similarity measure, and the resulting optimization problem.

A. Diffusion Models

Let $G=(V,E)$ be a directed graph with $|V|=n$ and $|E|=m$. Let $S \subseteq V$ be a fixed seed node set. In this paper, we consider a general stochastic diffusion model $\mathcal{D}$ in which a realization starting from S yields a random activated set $\mathcal{A}(S) \subseteq V$. As two classical examples that are widely studied in influence maximization [4], we review the *Linear Threshold (LT)* model [77], [78] and the *Independent Cascade (IC)* model [79], [80].

Under the LT model, each node $v \in V \setminus S$ is associated with a random threshold $\theta_v \in [0,1]$, typically sampled independently from the uniform distribution on $[0,1]$, and each directed edge $(u,v) \in E$ is assigned a nonnegative influence weight b_{uv} such that $\sum_{u:(u,v)\in E} b_{uv} \leq 1$. Starting from the seed node set S, the diffusion process unfolds in discrete steps. At each step, an inactive node v becomes active if the total influence weight it receives from its currently active in-neighbors is at least θ_v. Once a node becomes active, it remains active thereafter. The process continues until no further activation is possible, and one realization of the LT model uniquely determines a final activated set $\mathcal{A}(S) \subseteq V$.

Under the IC model, each directed edge $(u,v) \in E$ is associated with an activation probability $p_{uv} \in [0,1]$. Starting from the seed node set S, the diffusion process also unfolds in discrete steps. At each step, a node that becomes active in the current step gets a single chance to activate each currently inactive out-neighbor v, independently with probability p_{uv}. If the activation attempt succeeds, then v becomes active in the next step. Once a node becomes active, it remains active thereafter. The process continues until no further activation is possible, and one realization of the IC model uniquely determines a final activated set $\mathcal{A}(S) \subseteq V$.

B. Co-activation Similarity

Given the fixed seed set S, the diffusion process induces a pairwise similarity measure on V. For any pair of nodes $u, v \in V$, we define the co-activation similarity, i.e., the co-activation probability, as

$$W_{uv} \coloneqq \Pr(u \in \mathcal{A}(S) \wedge v \in \mathcal{A}(S)). \tag{1}$$

The value W_{uv} quantifies the strength of pairwise co-activation induced by diffusion from S, in the sense that a larger value

indicates that the two nodes are more likely to be activated together in the same realization. By definition, $W_{uv} = W_{vu}$, so the induced similarity is naturally symmetric. Collecting these values over all node pairs gives a matrix $W \in [0,1]^{n\times n}$.

However, in general, W is dense, and explicitly storing or processing all pairwise similarities would require $O(n^2)$ space. To obtain a sparse structure that retains only sufficiently strong similarities, we introduce a threshold parameter $\tau \in (0,1)$ and keep only the pairs whose co-activation similarity is at least τ. Formally, we define the thresholded similarity

$$W^{\tau}_{uv} \coloneqq W_{uv}\, \mathbb{I}[W_{uv} \geq \tau], \tag{2}$$

where $\mathbb{I}[\cdot]$ denotes the indicator function, which equals 1 if the condition holds and 0 otherwise. This thresholding step transforms the dense similarity matrix into a sparse weighted structure that will serve as the basis of our DCGS problem. Equivalently, we may focus only on the node pairs that survive thresholding, i.e.,

$$E^{\tau} \coloneqq \{\{u,v\} \subseteq V : W_{uv} \geq \tau, u \neq v\}. \tag{3}$$

Each element $\{u,v\} \in E^{\tau}$ is an unordered pair, i.e., $\{u,v\} = \{v,u\}$. Hence, E^{τ} can be viewed as the edge set of an undirected weighted graph induced by the diffusion process, where each retained pair $\{u,v\}$ carries weight W_{uv}.

C. Optimization Objective

We define the objective of DCGS on this thresholded similarity structure. For any nonempty node set $U \subseteq V$, let $E^{\tau}(U) \coloneqq \big\{\{u,v\} \in E^{\tau} : u \in U, v \in U\big\}$ be the set of retained pairs whose two endpoints both lie in U. That is, $E^{\tau}(U)$ consists of the strong co-activation relations induced by U. We then define the within-set co-activation weight of U as $W^{\tau}(U) \coloneqq \sum_{\{u,v\}\in E^{\tau}(U)} W_{uv}$, which measures the total strength of co-activation similarity contained inside U.

A natural objective would be to maximize $W^{\tau}(U)$. However, this would trivially favor very large node sets. To balance internal similarity and set size, we adopt the standard density objective from the densest-subgraph problem [14], and define $\rho(U) \coloneqq \frac{W^{\tau}(U)}{|U|}$. Intuitively, $\rho(U)$ measures the average amount of retained co-activation weight contributed per node in U. Hence, a set with high density is expected to contain a relatively concentrated group of nodes that are strongly co-activated with one another under diffusion from S.

In the above definition, we omit the weight subscript W^{τ} for brevity, i.e., $\rho(U) = \rho_{W^{\tau}}(U)$. When a different weight function $W' \neq W^{\tau}$ is used, we write the dependence explicitly as $\rho_{W'}(U)$.

The DCGS problem aims to find a nonempty node set $U^{\circ} \subseteq V \setminus S$ that maximizes this density, i.e., $U^{\circ} \in \arg\max_{\emptyset\neq U\subseteq V\setminus S} \rho(U)$. We exclude the seed set S and the co-activation weights between seed and non-seed nodes because $\rho(U)$ is intended to measure the internal co-activation cohesion among the discovered non-seed users. Since seed nodes are active initially, the co-activation probability between a seed node and a non-seed node reduces to the activation probability of the latter. Including such terms would therefore mix node-level activation likelihood with pairwise co-activation within the group. An alternative formulation that includes seed nodes is also possible, and our algorithmic framework remains applicable, but this paper focuses on the former interpretation. We define the corresponding optimal value as $OPT \coloneqq \rho(U^{\circ})$.

IV. Computational Challenges

The objective $\rho(U)$ is defined on the thresholded similarity structure induced by the co-activation probabilities W_{uv}. Before presenting our algorithm, we clarify the source of the computational difficulty. The difficulty does not primarily come from the downstream densest-subgraph optimization. Instead, it comes from the need to obtain sufficient information about the thresholded co-activation structure without explicitly computing all pairwise similarities.

We first consider the basic task of evaluating a single similarity value. Even for a fixed pair $u, v \in V$ with $u \neq v$, exact computation of W_{uv} is already intractable under the diffusion models considered in this paper.

Theorem 1. *Under either the LT model or the IC model, for any fixed $u, v \in V$ with $u \neq v$, computing the co-activation similarity W_{uv} in Eq.* (1) *exactly is #P-hard.*

Theorem 1 follows from the #P-hardness of exact influence computation under the LT and IC models, see, e.g., [15]. Therefore, an exact preprocessing strategy that explicitly computes all pairwise similarities is already ruled out at the level of a single entry.

This observation immediately extends to any exact approach that first materializes the full similarity matrix W and then derives the thresholded structure from it.

Corollary 2. *Under either the LT model or the IC model, any exact algorithm that constructs the full similarity matrix W is intractable in general. Consequently, any exact procedure that first computes W and then obtains W^{τ} or E^{τ} by thresholding is also intractable in general.*

At the same time, the challenge above is accompanied by a severe size issue. Let $n_0 \coloneqq |V \setminus S|$. The restriction of W to unordered pairs of non-seed nodes contains $\binom{n_0}{2}$ pairwise similarity values, so explicitly storing all such values would require $\Theta(n_0^2)$ space. Hence, a scalable algorithm should avoid both exact similarity computation and explicit materialization of the full pairwise structure. Nevertheless, the co-activation similarities can still be estimated from independent diffusion realizations by standard Monte Carlo sampling. This suggests a sampling-based route, provided that the downstream optimization remains tractable once the relevant thresholded weighted structure is available. Indeed, if the thresholded pair set E^{τ} and the corresponding weights $\{W_{uv} : \{u,v\} \in E^{\tau}\}$ were given, then our problem reduces to a standard weighted densest-subgraph problem.

Proposition 3. *If E^{τ} and its weight matrix W^{τ} are given exactly, then the DCGS problem can be solved in polynomial time.*

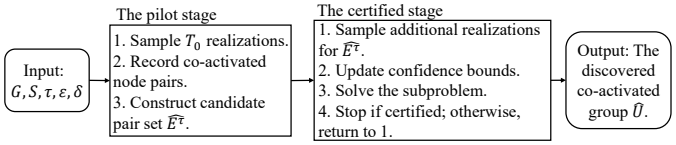


Fig. 2: Overview of the proposed algorithm for DCGS.

Proposition 3 follows from classical polynomial-time algorithms for the weighted densest-subgraph problem, see, e.g., [14], [18]. Therefore, the main algorithmic challenge in DCGS is not the densest-subgraph optimization itself. Rather, it is how to recover enough information about the thresholded co-activation structure from diffusion samples, while avoiding exact evaluation of all pairwise similarities and avoiding quadratic storage. This is precisely the motivation behind the two-stage sampling framework developed in Section V.

Remark 4. The hardness statements in Theorem 1, Corollary 2 and Proposition 3 concern exact computation. A related but potentially stronger question is the threshold decision problem, i.e., given a fixed threshold $\tau \in (0,1)$, deciding whether $W_{uv} \geq \tau$ for a designated pair $u \neq v$. From a complexity-theoretic viewpoint, this problem appears closely related to PP-type threshold problems [81], [82] for probabilistic reachability [83] and network reliability [84], [85]. However, we do not establish such a result in this paper. Instead, for the purpose of motivating our algorithmic design, it suffices to note that exact computation of W_{uv}, and hence any exact construction of the full similarity matrix W, is already intractable in general.

V. Methods

This section presents our two-stage algorithm for solving the DCGS problem. Let $n_0 \coloneqq |V \setminus S|$. As discussed in Section IV, a direct approach would require estimating and storing all pairwise similarities W_{uv} over the $\binom{n_0}{2}$ unordered pairs in $V \setminus S$, which is infeasible at scale due to the $\Theta(n_0^2)$ space requirement.

Our method avoids this bottleneck through a pilot stage followed by a certified stage. At a high level, the pilot stage samples a small number of diffusion realizations and tracks only the unordered node pairs that are co-activated in at least one observed realization. This yields a candidate set $\widehat{E}^{\tau} \subseteq \binom{V \setminus S}{2}$ that, with high probability, contains every pair $\{u,v\} \in E^{\tau}$, i.e., every pair with $W_{uv} \geq \tau$, while remaining much smaller than $\binom{n_0}{2}$ in typical instances.

Given $\widehat{E}^{\tau}$, the certified stage samples additional independent realizations, constructs checkpoint-wise confidence bounds for the tracked similarities, and interleaves estimation with optimization through an early-stopping rule. At each checkpoint, it invokes a densest-subgraph routine on both the current empirical weights and the optimistic upper-bound weights. The procedure terminates once the current solution is certified to satisfy the target approximation guarantee, and otherwise returns the final solution after a preset maximum number of sampled diffusion realizations.

Algorithm 1: Pilot Stage

Input: Graph $G = (V, E)$, diffusion model $\mathcal{D}$, seed set $S \subseteq V$, threshold $\tau \in (0,1)$, failure probability $\delta_0 \in (0,1)$

Output: Tracked pair set $\widehat{E}^{\tau} \subseteq \binom{V \setminus S}{2}$

1 $T_0 \leftarrow \left\lceil \frac{\ln(\binom{n_0}{2}/\delta_0)}{\ln(1/(1-\tau))} \right\rceil$;
2 $\widehat{E}^{\tau} \leftarrow \emptyset$;
3 **for** $t \leftarrow 1$ **to** T_0 **do**
4 Sample one diffusion realization under $\mathcal{D}$ on G from seeds S and obtain the activated node set $\mathcal{A}_t$;
5 **foreach** $u \in \mathcal{A}_t \setminus S$ **do**
6 **foreach** $v \in \mathcal{A}_t \setminus S$ **do**
7 **if** $u < v$ **then**
8 $\widehat{E}^{\tau} \leftarrow \widehat{E}^{\tau} \cup \{\{u,v\}\}$;
9 **return** $\widehat{E}^{\tau}$;

Figure 2 provides an overview of the proposed two-stage algorithm for DCGS. For clarity of presentation, we focus on the algorithmic procedure in this section, while deferring the theoretical analysis to Section VI.

A. Pilot Stage

The pilot stage constructs a candidate pair set $\widehat{E}^{\tau}$ that, with high probability, contains every pair $\{u,v\} \subseteq V \setminus S$ with $W_{uv} \geq \tau$, while typically remaining much smaller than the full pair universe $\binom{V \setminus S}{2}$. Consequently, the subsequent memory and estimation costs scale with $|\widehat{E}^{\tau}|$ rather than with all $\Theta(n_0^2)$ pairwise similarities. Algorithm 1 constructs $\widehat{E}^{\tau}$ by drawing T_0 independent diffusion realizations from seeds S and recording every unordered non-seed pair that is co-activated in at least one realization. Each realization can be generated by standard forward simulation under the adopted diffusion model $\mathcal{D}$ [4], while model-specific acceleration techniques [16], [86]–[88] can also be incorporated. The choice of T_0 in line 1 is justified in Section VI-A.

B. Certified Stage with Weighted Peeling

In the certified stage, we use weighted peeling as the default routine for solving the weighted densest-subgraph subproblems, as shown in Algorithm 2.

1) Weighted peeling: Given a weighted undirected graph on $V \setminus S$ with edge set $\widehat{E}^{\tau}$ and a nonnegative weight function W' on $\widehat{E}^{\tau}$, weighted peeling iteratively removes a node of minimum current weighted degree and returns the intermediate node set with the maximum density encountered during the process. Algorithm 2 implements this routine using a min-priority queue to maintain the current weighted degrees. It achieves a 2-approximation for the weighted densest-subgraph objective [17], and this guarantee is also used in the analysis of the certified stopping rule.

Algorithm 2: Weighted Peeling

Input: Node set V, seed set $S \subseteq V$, tracked pair set $\widehat{E}^{\tau} \subseteq \binom{V \setminus S}{2}$, nonnegative weights W on $\widehat{E}^{\tau}$
Output: Node set $U_{\max} \subseteq V \setminus S$

$U \leftarrow V \setminus S$;
foreach $u \in U$ **do**
  $d(u) \leftarrow \sum_{\{u,v\} \in \widehat{E}^{\tau},\ v \in U} W_{\{u,v\}}$;
Initialize a min-priority queue Q keyed by $d(u)$ for all $u \in U$;
$U_{\max} \leftarrow U$;
$\rho_{\max} \leftarrow \frac{1}{|U|} \sum_{\{u,v\} \in \widehat{E}^{\tau},\ u \in U,\ v \in U} W_{\{u,v\}}$;
while $|U| > 1$ **do**
  Extract a node $u^* \in \arg\min_{u \in U} d(u)$ from Q and remove it from U;
  foreach $\{u^*, v\} \in \widehat{E}^{\tau}$ *with* $v \in U$ **do**
    $d(v) \leftarrow d(v) - W_{\{u^*,v\}}$;
    Update the key of v in Q;
  $\rho(U) \leftarrow \frac{1}{|U|} \sum_{\{u,v\} \in \widehat{E}^{\tau},\ u \in U,\ v \in U} W_{\{u,v\}}$;
  if $\rho(U) > \rho_{\max}$ **then**
    $\rho_{\max} \leftarrow \rho(U)$;
    $U_{\max} \leftarrow U$;
return $U_{\max}$;

2) Certified sampling with early stopping: Given the tracked pair set $\widehat{E}^{\tau}$, the certified stage (Algorithm 3) estimates only the tracked co-activation similarities using additional independent diffusion realizations. For each $e \in \widehat{E}^{\tau}$, it maintains a co-activation counter C_e and constructs the thresholded empirical weight $\widetilde{W}_e(t)$ according to Eq. (2). To reduce overhead, confidence bounds are constructed only at geometrically increasing checkpoints.

At each checkpoint t, the algorithm forms lower and upper confidence weights $W^-(t)$ and $W^+(t)$, and invokes weighted peeling (Algorithm 2) to obtain $\widehat{U}(t)$ from $\widetilde{W}(t)$ and $U^+(t)$ from $W^+(t)$. The stopping rule compares the optimistic bound induced by $W^+(t)$ with the certified lower value of $\widehat{U}(t)$ under $W^-(t)$. If the condition in line 19 is satisfied, the algorithm returns $\widehat{U}(t)$; otherwise, sampling continues to the next checkpoint. The certified stage is capped by a preset sampling budget $T_{\max}$. If early stopping is never triggered, the algorithm returns the peeling solution computed from $\widetilde{W}(T_{\max})$. The budget $T_{\max}$ is chosen sufficiently large to ensure high-probability concentration of the empirical similarities. One valid choice is

$$T_{\max} := \left\lceil \frac{1}{2} \ln \frac{6|\widehat{E}^{\tau}|}{\delta_1} \cdot \max\left\{ \frac{1}{\gamma^2}, \frac{9(2+\varepsilon)^2}{\tau^2 \varepsilon^2} \cdot \min\left\{ (n_0 - 1)^2, |\widehat{E}^{\tau}|^2 \right\} \right\} \right\rceil, \tag{4}$$

where γ is a theoretical parameter introduced by a margin assumption (Assumption 8, which will be explained in Section VI-C). In practice, γ is not required by the early-stopping mechanism and is only used to obtain a closed-form worst-case cap $T_{\max}$. When a fixed cap is desired, we set γ to a small conservative value, e.g., $\gamma = 10^{-4}$, so that the resulting budget is sufficiently large in typical instances. Note that $T_{\max}$ depends on γ only when $\gamma < \frac{\tau\varepsilon}{3(2+\varepsilon)\min\{n_0 - 1, |\widehat{E}^{\tau}|\}}$. The justification of Eq. (4), together with the role of γ, is provided in Sections VI-B and VI-C.

Algorithm 3: Certified Stage

Input: Graph $G = (V, E)$, diffusion model $\mathcal{D}$, seed set $S \subseteq V$, tracked pair set $\widehat{E}^{\tau} \subseteq \binom{V \setminus S}{2}$, threshold $\tau \in (0, 1)$, approximation slack $\varepsilon > 0$, failure probability $\delta_1 \in (0, 1)$
Output: Node set $\widehat{U} \subseteq V \setminus S$

Set $T_{\max}$ by Eq. (4);
$t_{\text{check}} \leftarrow \left\lceil 2 \ln \frac{6|\widehat{E}^{\tau}|}{\delta_1} \right\rceil$;
$i \leftarrow 0$;
Initialize $C_e \leftarrow 0$ for all $e \in \widehat{E}^{\tau}$;
for $t \leftarrow 1$ **to** $T_{\max}$ **do**
  Sample one diffusion realization under $\mathcal{D}$ on G from seeds S and obtain the activated set $\mathcal{A}_t$;
  foreach $u \in \mathcal{A}_t \setminus S$ **do**
    foreach $v \in \mathcal{A}_t \setminus S$ **do**
      if $u < v$ **and** $\{u, v\} \in \widehat{E}^{\tau}$ **then**
        $C_{\{u,v\}} \leftarrow C_{\{u,v\}} + 1$;
  if $t = t_{\text{check}}$ **then**
    $r \leftarrow \sqrt{\frac{1}{2t} \ln \frac{3|\widehat{E}^{\tau}|(i+1)(i+2)}{\delta_1}}$;
    foreach $e \in \widehat{E}^{\tau}$ **do**
      $\widetilde{W}_e(t) \leftarrow \frac{C_e}{t} \cdot \mathbb{I}\left[\frac{C_e}{t} \geq \tau\right]$;
      $W_e^-(t) \leftarrow \left(\frac{C_e}{t} - r\right) \cdot \mathbb{I}\left[\frac{C_e}{t} - r \geq \tau\right]$;
      $W_e^+(t) \leftarrow \min\left\{1, \frac{C_e}{t} + r\right\} \cdot \mathbb{I}\left[\frac{C_e}{t} + r \geq \tau\right]$;
    $\widehat{U}(t) \leftarrow$ invoke Algorithm 2 on $(V, S, \widehat{E}^{\tau}, \widetilde{W}(t))$;
    $U^+(t) \leftarrow$ invoke Algorithm 2 on $(V, S, \widehat{E}^{\tau}, W^+(t))$;
    if $2\rho_{W^+(t)}(U^+(t)) \leq (2+\varepsilon)\rho_{W^-(t)}(\widehat{U}(t))$ **then**
      return $\widehat{U}(t)$;
    $t_{\text{check}} \leftarrow 2t_{\text{check}}$;
    $i \leftarrow i + 1$;
foreach $e \in \widehat{E}^{\tau}$ **do**
  $\widetilde{W}_e(T_{\max}) \leftarrow \frac{C_e}{T_{\max}} \cdot \mathbb{I}\left[\frac{C_e}{T_{\max}} \geq \tau\right]$;
$\widehat{U} \leftarrow$ invoke Algorithm 2 on $(V, S, \widehat{E}^{\tau}, \widetilde{W}(T_{\max}))$;
return $\widehat{U}$;

C. Putting It Together

Algorithm 4 coordinates the two stages by splitting the failure probability into two parts. We set $\delta_0 = \delta/2$ and

Algorithm 4: Main Algorithm for DCGS

Input: Graph $G = (V, E)$, diffusion model $\mathcal{D}$, seed set $S \subseteq V$, threshold $\tau \in (0, 1)$, approximation slack $\varepsilon > 0$, failure probability $\delta \in (0, 1)$
Output: Node set $\widehat{U} \subseteq V \setminus S$

$\delta_0, \delta_1 \leftarrow \delta/2$;
$\widehat{E}^\tau \leftarrow$ invoke Algorithm 1 on $(G, \mathcal{D}, S, \tau, \delta_0)$;
$\widehat{U} \leftarrow$ invoke Algorithm 3 on $(G, \mathcal{D}, S, \widehat{E}^\tau, \tau, \varepsilon, \delta_1)$;
return $\widehat{U}$;

Algorithm 5: Exact Densest-Subgraph Solver via Min-Cut

Input: Node set V, seed set $S \subseteq V$, tracked pair set $\widehat{E}^\tau \subseteq \binom{V \setminus S}{2}$, nonnegative weights W on $\widehat{E}^\tau$
Output: Node set $U \subseteq V \setminus S$

$\omega \leftarrow \sum_{e \in \widehat{E}^\tau} W_e$;
foreach $u \in V \setminus S$ **do**
  $d(u) \leftarrow \sum_{\{u,v\} \in \widehat{E}^\tau} W_{\{u,v\}}$;
foreach *candidate value* λ *in a search procedure* **do**
  Construct an s-t network $\mathcal{H}(\lambda)$ with node set $\{s, t\} \cup (V \setminus S)$;
  foreach $u \in V \setminus S$ **do**
    Add directed edge $s \to u$ with capacity ω;
    Add directed edge $u \to t$ with capacity $\omega + 2\lambda - d(u)$;
  foreach $\{u, v\} \in \widehat{E}^\tau$ **do**
    Add directed edges $u \to v$ and $v \to u$ with capacity $W_{\{u,v\}}$;
  Compute a minimum s-t cut (S_λ, T_λ) in $\mathcal{H}(\lambda)$;
  Update λ according to the search procedure;
Let λ^* be the final value returned by the search procedure and let (S^*, T^*) be the corresponding minimum cut;
$U \leftarrow S^* \setminus \{s\}$;
return U;

$\delta_1 = \delta/2$, run the pilot stage with failure budget δ_0 and the certified stage with failure budget δ_1, and output the set returned by the certified stage. By a union bound, the overall failure probability is at most δ. The performance guarantees implied by this composition are established in Section VI-D.

D. Certified Stage with Min-Cut

In this subsection, we present an exact variant that replaces weighted peeling with a min-cut-based solver, which yields optimal solutions to these weighted densest-subgraph subproblems.

Given $\widehat{E}^\tau$ and a nonnegative weight function W on $\widehat{E}^\tau$, we solve the corresponding weighted densest-subgraph problem on $V \setminus S$. This problem admits an exact polynomial-time solution via a standard reduction to a sequence of s-t minimum-cut computations [18]. Algorithm 5 implements this classical construction by building an auxiliary s-t network $\mathcal{H}$ whose cut structure encodes the density objective, and then recovering an optimal densest-subgraph solution from a minimum cut.

Line 4 in Algorithm 5 abstracts the search procedure over the density parameter λ and does not specify a particular strategy. In practice, we perform a binary search over λ with sufficient precision to recover an exact optimal solution. More sophisticated implementations are also possible, e.g., by integrating a dedicated parametric min-cut routine into Algorithm 5 [18], [89]. Since such routines are standard and orthogonal to our main contributions, we do not further elaborate on them in this paper.

Algorithm 6: Certified Stage with Min-Cut

Input: Same as Algorithm 3
Output: Node set $\widehat{U} \subseteq V \setminus S$

Invoke Algorithm 3, replacing line 1 by setting $T_{\max}$ as $T'_{\max}$ in Eq. (5), replacing lines 17, 18, and 25 by invoking Algorithm 5 instead of Algorithm 2, and replacing the stopping rule in line 19 by $\rho_{W^+(t)}(U^+(t)) \le (1+\varepsilon)\rho_{W^-(t)}(\widehat{U}(t))$;

Algorithm 7: Main Algorithm with Min-Cut

Input: Same as Algorithm 4
Output: Node set $\widehat{U} \subseteq V \setminus S$

Invoke Algorithm 4, replacing line 3 by invoking Algorithm 6 instead of Algorithm 3;

Algorithm 6 is identical to Algorithm 3 in its sampling procedure, checkpoint schedule, and confidence-bound construction. It differs in two respects. First, each invocation of weighted peeling is replaced by the exact min-cut-based solver in Algorithm 5 when solving the subproblems under the current weights, e.g., for $\widetilde{W}(t)$ and $W^+(t)$. Second, since the optimistic subproblem under $W^+(t)$ is solved exactly, the stopping rule can be tightened accordingly, which yields a $(1+\varepsilon)$-multiplicative criterion.

The fallback budget can also be adjusted accordingly. Under Assumption 8, a sufficient choice for the final budget is

$$T'_{\max} \coloneqq \left\lceil \frac{1}{2} \ln \frac{6|\widehat{E}^\tau|}{\delta_1} \cdot \max\left\{ \frac{1}{\gamma^2}, \frac{(1+\varepsilon)^2}{\tau^2 \varepsilon^2} \cdot \min\left\{ (n_0 - 1)^2, |\widehat{E}^\tau|^2 \right\} \right\} \right\rceil. \tag{5}$$

Finally, Algorithm 7 follows the same two-stage pipeline as Algorithm 4, except that it calls Algorithm 6 in the certified stage. This variant is useful when an exact densest-subgraph solution is preferred over the faster peeling routine, at the cost of higher per-checkpoint computation.

VI. Theoretical Analysis

This section provides the theoretical analysis of our algorithm. We first establish the correctness guarantees of the pilot

stage and the certified stage. We then justify the sampling budget $T_{\max}$ and analyze the overall complexity of the proposed algorithms. Proofs of all theoretical results are provided in the appendix.

A. Correctness of the Pilot Stage

We first establish that the pilot stage retains all pairs in E^τ with high probability.

Lemma 5. *Fix $\tau \in (0,1)$ and $\delta_0 \in (0,1)$. Sample T_0 independent realizations from seeds S, yielding activated sets $\mathcal{A}_1, \dots, \mathcal{A}_{T_0}$. If $T_0 \geq \frac{\ln\left(\binom{n_0}{2}/\delta_0\right)}{\ln(1/(1-\tau))}$, then $\Pr(E^\tau \subseteq \widehat{E}^\tau) \geq 1-\delta_0$.*

Consequently, the certified stage only needs to estimate similarities on $\widehat{E}^\tau$ rather than considering the full pair universe $\binom{V\setminus S}{2}$.

B. Correctness of the Certified Stage

We next establish the simultaneous validity of the checkpoint-wise confidence bounds used by Algorithm 3.

Lemma 6. *Fix $\widehat{E}^\tau \subseteq \binom{V\setminus S}{2}$, $\tau \in (0,1)$, $T_{\max} \in \mathbb{N}$, and $\delta_1 \in (0,1)$. Let $\mathcal{T}$ be the checkpoint set induced by t_{check} in Algorithm 3, and let $r(t)$ be the radius computed at checkpoints, i.e., r in line 12. Define $W_e^-(t)$ and $W_e^+(t)$ as in lines 15–16. Then, with probability at least $1 - 2\delta_1/3$, simultaneously for all $e \in \widehat{E}^\tau$ and all $t \in \mathcal{T}$,*

$$W_e^-(t) \leq W_e \cdot \mathbb{I}[W_e \geq \tau] \leq W_e^+(t). \tag{6}$$

Thus, on a common high-probability event, $W^-(t)$ and $W^+(t)$ sandwich the true thresholded weights at every checkpoint, forming the basis of the certified stopping rule.

We next establish the approximation guarantee of the early-stopping rule by combining the monotonicity of the density objective, the 2-approximation guarantee of weighted peeling, and the optimistic and pessimistic objective bounds.

Theorem 7. *Fix a checkpoint $t \in \mathcal{T}$. Assume $E^\tau \subseteq \widehat{E}^\tau$ and $W_e^-(t) \leq W_e^\tau \leq W_e^+(t)$ for all $e \in \widehat{E}^\tau$. If the stopping rule in line 19 holds, i.e.,*

$$2 \cdot \rho_{W^+(t)}\big(U^+(t)\big) \leq (2+\varepsilon) \cdot \rho_{W^-(t)}\big(\widehat{U}(t)\big), \tag{7}$$

then

$$(2+\varepsilon) \cdot \rho\big(\widehat{U}(t)\big) \geq OPT. \tag{8}$$

Thus, once the stopping rule is triggered, the certified stage may safely return $\widehat{U}(t)$.

C. Justification of $T_{\max}$

We next justify the choice of $T_{\max}$ in Eq. (4). Throughout this subsection, we condition on the pilot success event $E^\tau \subseteq \widehat{E}^\tau$, so that no pair in E^τ is missed by the certified stage.

Recall that at $t = T_{\max}$, Algorithm 3 maintains counts C_e for $e \in \widehat{E}^\tau$ in line 10 and forms the thresholded empirical weights $\widetilde{W}_e(T_{\max})$ in line 24. Define the empirical mean $\widehat{W}_e := C_e/T_{\max}$ and the radius

$$r := \sqrt{\frac{1}{2T_{\max}} \ln \frac{6|\widehat{E}^\tau|}{\delta_1}}. \tag{9}$$

A standard concentration argument, given in the appendix, implies that with probability at least $1 - \delta_1/3$, simultaneously for all $e \in \widehat{E}^\tau$,

$$\big|\widehat{W}_e - W_e\big| \leq r. \tag{10}$$

To control the thresholding step at $t = T_{\max}$, we introduce the following margin assumption.

Assumption 8. There exists a constant $\gamma > 0$ such that for all $e \in \widehat{E}^\tau$, $W_e \leq \tau - \gamma$ or $W_e \geq \tau + \gamma$.

Remark 9. Assumption 8 excludes the degenerate case in which the threshold τ is critical on the tracked set, i.e., $\tau = W_e$ for some $e \in \widehat{E}^\tau$. Since the tracked set is finite, an arbitrarily small perturbation of τ typically avoids this critical case without changing the thresholded structure on $\widehat{E}^\tau$. Moreover, γ is only used to justify the worst-case fallback guarantee at $t = T_{\max}$, and is not required by the early-stopping mechanism itself. A more detailed discussion is deferred to the appendix.

Let $L := \min\{n_0 - 1, |\widehat{E}^\tau|\}$. Under Assumption 8, the edgewise concentration bound in Eq. (10) can be lifted to a uniform additive error bound on the density objective. The following lemma formalizes this step.

Lemma 10. *Assume $E^\tau \subseteq \widehat{E}^\tau$ and Assumption 8. On the event in Eq.* (10)*, if $r \leq \gamma$, then for every nonempty $U \subseteq V \setminus S$,*

$$\big|\rho_{\widetilde{W}(T_{\max})}(U) - \rho(U)\big| \leq \frac{r}{2} L. \tag{11}$$

We can now combine Lemma 10 with the standard 2-approximation guarantee of weighted peeling to obtain the final fallback guarantee at $t = T_{\max}$.

Theorem 11. *Assume $E^\tau \subseteq \widehat{E}^\tau$ and Assumption 8. Let $\widehat{U}$ be the output in line 26 of Algorithm 3. On the event in Eq.* (10)*, if $r \leq \gamma$ and $\frac{r}{2}L \leq \frac{\varepsilon\tau}{6(2+\varepsilon)}$, then*

$$(2+\varepsilon)\rho(\widehat{U}) \geq OPT. \tag{12}$$

To apply Theorem 11, it suffices to choose $T_{\max}$ so that the radius r in Eq. (9) satisfies the two conditions $r \leq \gamma$ and $\frac{r}{2}L \leq \frac{\varepsilon\tau}{6(2+\varepsilon)}$. Substituting Eq. (9) into these inequalities and rearranging them yields a sufficient lower bound on $T_{\max}$, which is exactly Eq. (4).

D. Overall Guarantee of the Main Algorithm

We now summarize the approximation guarantee of Algorithm 4 by combining the preceding results.

Theorem 12. *Fix $\tau \in (0,1)$, $\varepsilon > 0$, and $\delta \in (0,1)$. Let $\widehat{U}$ be the output of Algorithm 4. With probability at least $1 - \delta$,*

$$(2+\varepsilon)\rho(\widehat{U}) \geq OPT \tag{13}$$

holds under either of the following conditions:

1) *Algorithm 3 terminates early at some checkpoint $t \in \mathcal{T}$ by the stopping rule in line 19, returning $\widehat{U}(t)$ in line 20.*
2) *Algorithm 3 does not terminate early and returns $\widehat{U}$ in line 26, provided that Assumption 8 holds.*

The guarantee in Theorem 12 depends on how Algorithm 3 terminates. If it stops early at a checkpoint, Eq. (13) follows

from Theorem 7 and does not require Assumption 8. If it reaches $t = T_{\max}$, then Eq. (13) follows from Theorem 11, and thus additionally requires Assumption 8.

Remark 13. Without a positive margin (Assumption 8), the worst-case approximation guarantee of the fallback at $t = T_{\max}$ can fail. A simple counterexample is given in the appendix.

The same argument also applies to Algorithm 7. Replacing weighted peeling by the exact densest-subgraph solver removes the factor 2 from the subproblem approximation, so under the corresponding stopping rule, the output $\widehat{U}$ of Algorithm 7 satisfies $(1+\varepsilon)\rho(\widehat{U}) \geq OPT$. This holds when Algorithm 6 terminates early at some checkpoint, and also for the fallback at $t = T'_{\max}$ under Assumption 8.

E. Complexity Analysis

Let $\hat{m} := |\widehat{E}^{\tau}|$. We consider worst-case complexity and assume that one diffusion realization under $\mathcal{D}$ can be simulated in $\mathcal{O}(m)$ time. Since $|\mathcal{A}_t \setminus S| \leq n_0 \leq n$, enumerating all unordered pairs in $\mathcal{A}_t \setminus S$ costs $\mathcal{O}(n^2)$ per realization in the worst case.

The pilot stage runs T_0 realizations. Hence, its worst-case time is $\mathcal{O}(T_0(m+n^2))$, and its additional space is $\mathcal{O}(\hat{m})$ for storing $\widehat{E}^{\tau}$.

The certified stage runs at most $T_{\max}$ realizations. For counter updates, it is sufficient to mark the activated nodes and scan the tracked set $\widehat{E}^{\tau}$, which yields $\mathcal{O}(\hat{m})$ update time per realization. Therefore, the worst-case sampling and bookkeeping time is $\mathcal{O}(T_{\max} \cdot (m+\hat{m}))$, and the space usage is $\mathcal{O}(\hat{m})$.

Since checkpoints grow geometrically, there are $\mathcal{O}(\log T_{\max})$ checkpoints. At each checkpoint, the algorithm solves two weighted densest-subgraph subproblems. With weighted peeling, this contributes $\mathcal{O}\big((\log T_{\max})(n_0+\hat{m})\log n_0\big)$ time over all checkpoints.

Combining the three parts yields the worst-case time complexity $\mathcal{O}((m+n^2)T_0 + (m+\hat{m})T_{\max} + (n_0+\hat{m})\log n_0 \cdot \log T_{\max})$. For constant τ, we have $T_0 = \mathcal{O}(\log(n/\delta))$. Moreover, in the regime where the γ-term does not dominate Eq. (4), we have $T_{\max} = \mathcal{O}\big(\frac{1}{\varepsilon^2}\min\{n_0^2, \hat{m}^2\}\log\frac{\hat{m}}{\delta}\big)$. In particular, if $m = \Theta(n)$ and $\hat{m} = \Theta(n)$, then the time complexity becomes $\mathcal{O}\Big(\frac{n^3}{\varepsilon^2}\log\frac{n}{\delta}\Big)$.

For the min-cut variant, we analyze Algorithm 5 when it is instantiated by the classical parametric min-cut method of [18]. The auxiliary network has $N = n_0 + 2$ nodes and $M = 2\hat{m} + 2n_0$ directed edges, and one call costs $\mathcal{O}(N \cdot T_{\text{flow}}(N,M))$ time, where $T_{\text{flow}}(N,M)$ denotes the runtime of a single max-flow computation on a graph with N nodes and M edges. Under the bound $T_{\text{flow}}(N,M) = \mathcal{O}(MN)$ for exact max-flow computation [90], one call to Algorithm 5 costs $\mathcal{O}(MN^2)$. Since there are two such calls per checkpoint, the total checkpoint cost is $\mathcal{O}\big((\log T_{\max})MN^2\big) = \mathcal{O}\big((\log T_{\max})(\hat{m}+n_0)n_0^2\big)$. Hence, the overall worst-case time of the min-cut variant is $\mathcal{O}\big((m+n^2)T_0 + (m+\hat{m})T_{\max} + (\hat{m}+n_0)n_0^2 \cdot \log T_{\max}\big)$.

Finally, all bounds above correspond to the worst-case scenario in which the certified stage does not terminate early and runs until $t = T_{\max}$. In practice, the early-stopping rule often terminates the certified stage much earlier, which substantially reduces the runtime. This is supported by the experimental results in Section VII.

VII. Experiments

Our experiments are designed to answer the following questions: (1) How effectively do the proposed algorithms and baselines optimize the DCGS objective? (2) How do the proposed algorithms compare with the baselines in terms of running time and memory usage? (3) How do the proposed algorithms and baselines perform under different parameter settings? (4) How does the DCGS objective differ from other related optimization objectives?

A. Experimental Setup

All algorithms are implemented in `C++17` and compiled with the `-O3` flag on a Linux server. The source code is available at https://anonymous.4open.science/r/dcgs-VLDB.

a) Datasets: We use eleven real-world OSN datasets spanning multiple orders of magnitude in graph size. Their basic statistics are reported in the first four columns of Table I (sorted by peak memory usage). Among them, *Hamsterster* and *Douban* are obtained from the Network Repository [91], while the remaining datasets are collected from SNAP [92]. Since DCGS is defined on directed graphs, we preprocess each originally undirected graph by replacing every undirected edge with a pair of reciprocal directed edges.

b) Diffusion Models and Parameters: We consider the two classical diffusion models introduced in Section III-A, namely, the LT model [77], [78] and the IC model [79], [80]. Unless otherwise specified, the LT model is used as the default diffusion model in our experiments. This choice is motivated by the observation that the LT model captures activation as the cumulative effect of multiple in-neighbors, which aligns more naturally with the co-activation perspective underlying DCGS. In contrast, the IC model characterizes diffusion as a collection of independent edge-level propagation events. For the LT model, we set $b_{uv} = \frac{c}{\text{InDeg}(v)}$ for each edge (u,v), where $\text{InDeg}(v)$ is the in-degree of v, and use $c = 1/2$ by default. For the IC model, we use the analogous setting $p_{uv} = \frac{c}{\text{InDeg}(v)}$. For the seed set S, we use the 10 nodes with the largest out-degrees by default. We use $\varepsilon = 0.5$, $\delta = 10^{-3}$, $\gamma = 10^{-4}$, and $\tau = 0.2$ as the default parameter values. In all completed experimental runs of the proposed algorithms, the certified stage terminated by the early-stopping criterion before reaching $T_{\max}$ for *DCGS-P* or $T'_{\max}$ for *DCGS-MC*. Therefore, $\gamma = 10^{-4}$ only determines the unused worst-case fallback budget and does not affect the reported stopping points or outputs.

c) Compared methods: The following methods are included in our experimental comparison. Since DCGS is a new problem, there are no existing methods designed specifically for it. Therefore, the compared baselines in our experiments

TABLE I: Dataset statistics and performance comparison under the default parameter setting (sorted by peak memory usage).

Dataset	n	m	Original Type	Obj. Value	Runtime (s)			Peak Mem. (MB)		
					DCGS-P	*DCGS-MC*	*Greedy*	*DCGS-P*	*DCGS-MC*	*Greedy*
Hamsterster	2,426	33,260	Undirected	0.625	75.6	26.2	**3.20**	**27.7**	54.1	383
Facebook	4,039	176,468	Undirected	6.14	44.4	111	**16.7**	**132**	279	773
DBLP	317,080	2,099,732	Undirected	2.51	87.0	186	**23.5**	**261**	559	3,571
Douban	154,908	654,324	Undirected	8.64	197	567	**52.7**	**518**	1,329	13,837
Twitter	81,306	1,768,135	Directed	22.5	20,493	5,071	**705**	**4,207**	11,250	67,601
Google+	107,614	13,673,453	Directed	16.4	4,259	5,430	**754**	**5,086**	13,388	86,189
Email-EuAll	265,009	418,956	Directed	477	2,693	4,848	**2,133**	**5,282**	13,661	51,206
Epinions	75,879	508,837	Directed	58.0	2,690	7,110	**1,019**	**5,802**	15,983	96,856
Pokec	1,632,803	30,622,564	Directed	66.2	**10,226**	22,023	—	**21,030**	61,058	OOM
Orkut	3,072,441	234,370,166	Undirected	46.9	**34,008**	—	—	**73,867**	OOM	OOM
LiveJournal	4,846,609	68,475,391	Directed	67.0	**57,584**	—	—	**124,600**	OOM	OOM

Note. Under the default setting, all six compared algorithms attain identical objective values to the reported precision on each dataset. In addition, the runtime and peak memory usage of *LS*, *CDR*, and *APR* are omitted for brevity, since they are nearly identical to those of *Greedy* on the same dataset, as all four methods are dominated by the same fixed budget of 10,000 diffusion realizations. OOM means that the memory usage exceeds the limit of $200\,\mathrm{GB} = 204{,}800\,\mathrm{MB}$.

are either adapted from related algorithms or constructed as simple intuitive heuristics.

DCGS+Peeling (DCGS-P): The proposed peeling-based algorithm for DCGS (Algorithm 4).

DCGS+MinCut (DCGS-MC): The proposed min-cut-based algorithm for DCGS (Algorithm 7).

Greedy: An add-only heuristic inspired by Tsourakakis et al. [93]. It estimates pairwise co-activation similarities from a prescribed number of diffusion realizations, initializes the solution with the highest-similarity pair, and then iteratively adds the node that maximally improves the estimated objective until no further improvement is possible. In this sense, it follows a greedy strategy opposite to peeling, growing the solution from a small initial set rather than shrinking it from the full node set.

Local Search (LS): A local-search heuristic initialized by *Greedy*. It repeatedly applies the best improving operation among node addition, node removal, and node swap, and stops when no improving operation exists or when the total number of attempts reaches 1,000. This method is inspired by the refinement framework of Lee and Lakshmanan [94] and adapted to the DCGS setting.

Co-Activation Degree Ranking (CDR): A ranking-based heuristic that estimates pairwise co-activation similarities from diffusion realizations, scores each node by the total estimated similarity of its above-threshold incident pairs, ranks nodes accordingly, and returns the prefix with the largest estimated objective value.

Activation Probability Ranking (APR): A ranking-based heuristic that estimates node activation frequencies from diffusion realizations, ranks nodes by these estimates, and returns the prefix with the largest estimated objective value.

For *Greedy*, *LS*, *CDR*, and *APR*, the prescribed number of diffusion realizations is set to 10,000, following [95]. This practical fixed budget is far smaller than the sample size that would be required under a direct high-probability analysis based on Hoeffding's inequality [96] together with a union bound over the relevant pairwise estimates. If equipped with such statistically rigorous parameter settings, these methods would incur prohibitively high running time and would no longer be executable on large graphs in practice.

A major limitation of these four methods is their space complexity, since all of them require storing an almost full co-activation matrix and thus become impractical on large graphs or with large seed sets. In our experiments, the memory limit is set to $200\,\mathrm{GB}$. Following [19], each returned solution is evaluated under the DCGS objective using 10,000 fresh diffusion realizations sampled independently of those used during optimization. All results are averaged over five independent runs to mitigate randomness.

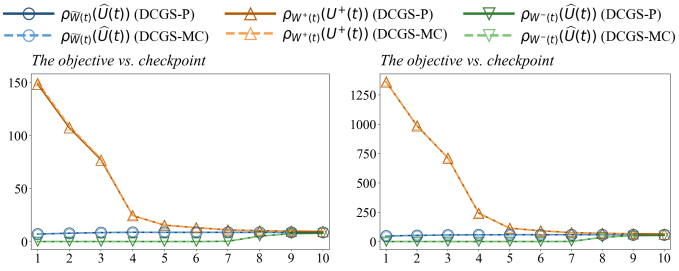
At the implementation level, the compared methods use consistent low-level data representations and comparable storage schemes. Additional details are provided in the appendix.

B. Experimental Results

Due to space limitations, from Section VII-B2 onward, we report results only on two representative datasets, *Douban* and *Epinions*.

1) Effectiveness: We first evaluate the effectiveness of the compared methods in optimizing the DCGS objective. The results are reported in the objective value column of Table I. Under the default parameter setting, all compared methods achieve the same objective value on each dataset, with virtually zero variance across the five runs. For this reason, Table I reports only one objective value per dataset. These results indicate that the proposed algorithms are effective in optimizing the DCGS objective and achieve solution quality comparable to that of the baselines.

2) Efficiency: We next compare the running time and peak memory usage of the evaluated methods, as reported in Table I. Overall, *DCGS-P* uses the least memory among the reported methods, while *DCGS-MC* also remains substantially more memory-efficient than *Greedy*, whose almost full co-activation matrix causes its memory usage to grow rapidly with graph size and become infeasible on large datasets. Although the pilot stage may enumerate $\Theta(n_0^2)$ co-activated pairs and $\widehat{E}^{\tau}$ may itself have quadratic size in the worst case, this regime is not approached on the evaluated datasets. In practice, the proposed methods exhibit substantially lower memory



(a) *Douban* (b) *Epinions*

Fig. 3: Estimated objective value and its upper and lower bounds at different checkpoints.

TABLE II: Approximation ratios at different checkpoints.

Dataset	Method	Checkpoint					
		t_1–t_5	t_6	t_7	t_8	t_9	t_{10}
Douban	*DCGS-P*	$+\infty$	$+\infty$	99.1	4.04	2.61	2.41
	DCGS-MC	$+\infty$	$+\infty$	49.6	2.02	1.31	—
Epinions	*DCGS-P*	$+\infty$	84,475	198	3.93	2.64	2.42
	DCGS-MC	$+\infty$	42,237	98.9	1.96	1.32	—

consumption, while the pilot-stage overhead remains small as reported in the full version. Similar runtime and memory results apply to *LS*, *CDR*, and *APR*, and are therefore omitted.

In terms of running time, the baselines are faster under their fixed sampling budget, whereas the proposed algorithms incur additional cost to provide high-probability accuracy and approximation guarantees. This is unsurprising, because the baselines rely on a prescribed number of diffusion realizations without providing high-probability accuracy guarantees for the co-activation similarities. Under a more stringent sampling requirement that ensures such estimates are simultaneously accurate with high probability, these methods would require far more samples and would become impractical on large graphs [4], [97]. *DCGS-P* is generally faster than *DCGS-MC*. The higher computational cost of *DCGS-MC* mainly arises from the repeated min-cut computations at the checkpoints, whereas the weighted-peeling subproblems in *DCGS-P* incur substantially lower optimization overhead. Overall, these results demonstrate that the proposed algorithms, especially *DCGS-P*, scale much better than the heuristic baselines because they avoid materializing an almost complete co-activation matrix. Detailed runtime breakdowns by stage are provided in the full version.

3) Approximation ratio: To better understand how the approximation ratio evolves during the certified stage, we report the results in Figure 3 and Table II. Figure 3 shows how the estimated objective value and its upper and lower bounds change across checkpoints for the two proposed methods. Table II reports the corresponding approximation ratios. At the early checkpoints, the upper bound is very loose and the lower bound remains zero, so the approximation ratio is not yet informative. As the number of samples increases, the lower bound becomes positive and the upper bound becomes much tighter. Starting from around the seventh checkpoint, the approximation ratio becomes meaningful and then quickly stabilizes over the following checkpoints.

TABLE III: Performance comparison at the stopping point for the 2.5-approximation criterion.

Dataset	Method	Runtime (s)	Peak Mem. (MB)
Douban	*DCGS-P*	197	518
	DCGS-MC	422	1,329
Epinions	*DCGS-P*	2,690	5,802
	DCGS-MC	5,702	15,983

4) Parameter sensitivity: We next examine the sensitivity of the proposed methods to different parameter settings.

Varying τ. As shown in Figure 4, decreasing τ increases the memory usage of both *DCGS-P* and *DCGS-MC*, because more co-activation pairs must be retained. *DCGS-P* is less sensitive to decreases in τ, whereas the memory usage of *DCGS-MC* grows more rapidly. Nevertheless, both algorithms remain feasible throughout the evaluated range, including the smallest value of τ. A similar trend is observed for runtime. Reducing τ generally improves the objective value by retaining additional pairs with relatively small co-activation similarities (see the appendix). However, the absolute improvement remains modest over the evaluated range. These results indicate that τ controls a practical tradeoff between objective quality and computational cost.

Varying ε. Figure 5 shows that increasing ε noticeably reduces the runtime of *DCGS-P*, but has a much smaller effect on *DCGS-MC*. This is consistent with Section VII-B2. *DCGS-P* is mainly dominated by certified-stage sampling, whereas *DCGS-MC* incurs a much higher optimization cost. By contrast, ε has little effect on memory usage.

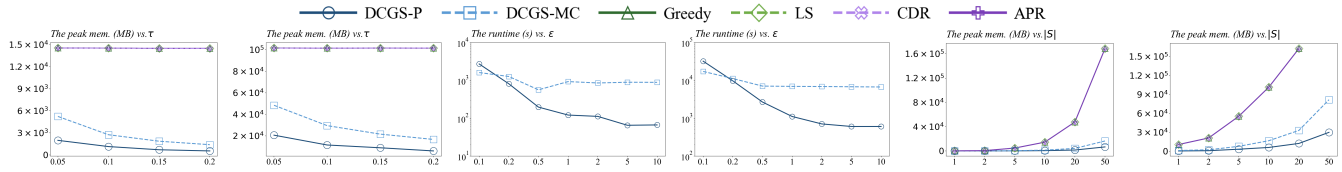
Varying δ. Increasing δ slightly reduces both runtime and peak memory usage, as a larger δ corresponds to a looser confidence requirement. Overall, its effect is moderate for both methods.

Varying S. Figure 6 shows the results when S is constructed by selecting the top-$|S|$ nodes ranked by out-degree. As $|S|$ increases, the memory usage of the baselines grows dramatically because larger seed sets activate more non-seed nodes and produce more co-activation pairs that must be stored. In particular, when $|S| = 50$, the baseline methods already become impractical even on medium-scale graphs. This further demonstrates the scalability of the proposed framework with respect to the seed-set size.

Varying $\mathcal{D}$. We also replace the default LT model with the IC model. Although the absolute objective values and running statistics may vary, the overall trends remain consistent with those under the default LT setting.

Overall, the proposed methods exhibit stable behavior under different parameter settings. Across these settings, *DCGS-P* remains the more practical choice in terms of efficiency and scalability.

5) Comparison under the same approximation target: We also compare *DCGS-P* and *DCGS-MC* under the same target approximation ratio, i.e., 2.5. Specifically, we keep the default setting for *DCGS-P* and set $\varepsilon = 1.5$ for *DCGS-MC*. Table III shows that *DCGS-MC* still incurs higher runtime and peak



(a) *Douban* (b) *Epinions*

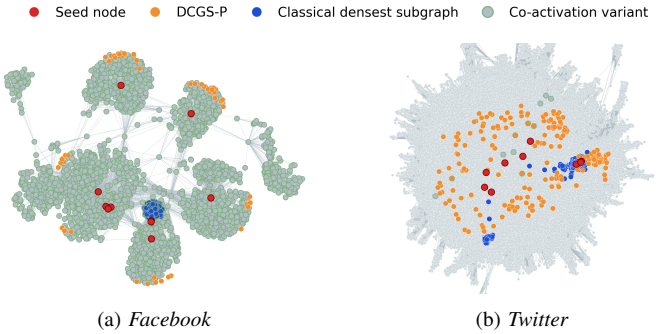
Fig. 4: Peak memory usage under different values of τ.

(a) *Douban* (b) *Epinions*

Fig. 5: Runtime under different values of ε.

(a) *Douban* (b) *Epinions*

Fig. 6: Peak memory usage under different sizes of S.



(a) *Facebook* (b) *Twitter*

Fig. 7: Comparison of the node sets returned by different objectives on *Facebook* and *Twitter*.

memory usage than *DCGS-P*, and the overall trend remains consistent with Section VII-B2.

6) Distinctiveness of the DCGS objective: We compare the solution returned by *DCGS-P* with those obtained under two related objectives, namely, the classical densest-subgraph objective and a structure-constrained co-activation variant inspired by [61]. In the latter variant, co-activation weights are counted only for node pairs that are adjacent in the original graph, after which weighted peeling is applied to the resulting constrained graph.

As shown in Figure 7, these three objectives lead to qualitatively different results. The classical densest-subgraph objective identifies a tightly connected subgraph in the original network. By contrast, DCGS may return nodes that are distributed across different parts of the graph but are frequently co-activated under diffusion. This behavior is consistent with our motivation of identifying nodes that may be structurally distant yet likely to be activated together in the same campaign. The structure-constrained co-activation variant, on the other hand, tends to select a node set that covers almost the entire graph on *Facebook* (3,979 out of 4,039 nodes), whereas it selects only a few nodes on *Twitter* (11 out of 81,306 nodes). The strong contrast between the two datasets suggests that imposing the original graph structure as a hard constraint yields solutions that are highly structure-sensitive and potentially unstable across instances. Overall, DCGS defines a distinct group-discovery objective and produces substantially different solutions from structure-based formulations.

7) Ablation study: We further conduct an ablation study to disentangle the practical roles of pilot screening and statistical certification. Specifically, we consider an uncertified variant that keeps the pilot stage unchanged but replaces the certified stage with $10{,}000$ additional diffusion realizations followed by weighted peeling. On both *Douban* and *Epinions*, this variant attains the same objective values as *DCGS-P* to the reported precision. Relative to *DCGS-P*, it reduces runtime by 52.7% on *Douban* and 66.4% on *Epinions*, while increasing peak memory usage by only 12.9% and 22.5%, respectively. It still reduces peak memory usage by 95.8% on *Douban* and 92.7% on *Epinions* compared with *Greedy*. These results show that the pilot stage drives most of the empirical memory savings, while the certified stage converts the resulting fixed-budget heuristic into an algorithm with explicit high-probability approximation guarantees. Detailed results are provided in the appendix.

VIII. Discussion and Future Work

DCGS assumes a fixed seed set and a prescribed threshold τ. Natural extensions include optimizing the seed set, developing data-dependent strategies for selecting τ, and considering more general diffusion models. Memory-efficient pair-discovery techniques, such as streaming heavy-hitter or sketching-based methods, are also complementary to our framework and could potentially be incorporated into the pilot stage to further reduce the cost of candidate-pair discovery. Our experiments further reveal a gap between the sample budget sufficient for obtaining high-quality empirical solutions and that required for statistical certification, motivating more sample-efficient certification methods. Finally, since the proposed framework maintains a certified approximation ratio at each checkpoint, it can naturally operate as an anytime algorithm, where alternative stopping policies may balance solution quality, theoretical guarantees, and computational budget.

IX. Conclusion

In this paper, we introduced the DCGS problem for discovering densely co-activated groups under diffusion from a given seed set. To solve DCGS, we developed a two-stage algorithmic framework that combines pilot screening, certified sampling, and densest-subgraph optimization, with two implementations. We further established theoretical approximation guarantees for the proposed methods. Extensive experiments on real-world datasets demonstrated their effectiveness and scalability.

REFERENCES

[1] W. Chen, C. Castillo, and L. V. Lakshmanan, *Information and influence propagation in social networks*. Morgan & Claypool Publishers, 2013.

[2] E. Bakshy, I. Rosenn, C. Marlow, and L. Adamic, "The role of social networks in information diffusion," in *Proceedings of the 21st International Conference on World Wide Web*, ser. WWW '12. New York, NY, USA: Association for Computing Machinery, 2012, p. 519–528. [Online]. Available: https://doi.org/10.1145/2187836.2187907

[3] E. Rogers, *Diffusion of Innovations, 5th Edition*. Free Press, 2003. [Online]. Available: https://books.google.co.jp/books?id=9U1K5LjUOwEC

[4] D. Kempe, J. Kleinberg, and E. Tardos, "Maximizing the spread of influence through a social network," in *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '03. New York, NY, USA: Association for Computing Machinery, 2003, p. 137–146. [Online]. Available: https://doi.org/10.1145/956750.956769

[5] T. W. Valente, "Network models of the diffusion of innovations," *Computational & Mathematical Organization Theory*, vol. 2, no. 2, pp. 163–164, 1996.

[6] A. Guille, H. Hacid, C. Favre, and D. A. Zighed, "Information diffusion in online social networks: a survey," *SIGMOD Rec.*, vol. 42, no. 2, p. 17–28, Jul. 2013. [Online]. Available: https://doi.org/10.1145/2503792.2503797

[7] M. Gomez-Rodriguez, L. Song, N. Du, H. Zha, and B. Schölkopf, "Influence estimation and maximization in continuous-time diffusion networks," *ACM Trans. Inf. Syst.*, vol. 34, no. 2, Feb. 2016. [Online]. Available: https://doi.org/10.1145/2824253

[8] B. Lucier, J. Oren, and Y. Singer, "Influence at scale: Distributed computation of complex contagion in networks," in *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 735–744. [Online]. Available: https://doi.org/10.1145/2783258.2783334

[9] Y. Li, J. Fan, Y. Wang, and K.-L. Tan, "Influence maximization on social graphs: A survey," *IEEE Transactions on Knowledge and Data Engineering*, vol. 30, no. 10, pp. 1852–1872, 2018.

[10] M. Jaouadi and L. Ben Romdhane, "A survey on influence maximization models," *Expert Systems with Applications*, vol. 248, p. 123429, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S095741742400294X

[11] N. P. Nguyen, G. Yan, M. T. Thai, and S. Eidenbenz, "Containment of misinformation spread in online social networks," in *Proceedings of the 4th Annual ACM Web Science Conference*, ser. WebSci '12. New York, NY, USA: Association for Computing Machinery, 2012, p. 213–222. [Online]. Available: https://doi.org/10.1145/2380718.2380746

[12] S. Raponi, Z. Khalifa, G. Oligeri, and R. Di Pietro, "Fake news propagation: A review of epidemic models, datasets, and insights," *ACM Trans. Web*, vol. 16, no. 3, Sep. 2022. [Online]. Available: https://doi.org/10.1145/3522756

[13] A. Tong, D.-Z. Du, and W. Wu, "On misinformation containment in online social networks," in *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., vol. 31. Curran Associates, Inc., 2018. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2018/file/9b04d152845ec0a378394003c96da594-Paper.pdf

[14] A. V. Goldberg, "Finding a maximum density subgraph," *University of California Berkeley, Technical Report*, 1984.

[15] W. Chen, C. Wang, and Y. Wang, "Scalable influence maximization for prevalent viral marketing in large-scale social networks," in *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '10. New York, NY, USA: Association for Computing Machinery, 2010, p. 1029–1038. [Online]. Available: https://doi.org/10.1145/1835804.1835934

[16] C. Borgs, M. Brautbar, J. Chayes, and B. Lucier, *Maximizing Social Influence in Nearly Optimal Time*, 2014, pp. 946–957. [Online]. Available: https://epubs.siam.org/doi/abs/10.1137/1.9781611973402.70

[17] M. Charikar, "Greedy approximation algorithms for finding dense components in a graph," in *International workshop on approximation algorithms for combinatorial optimization*. Springer, 2000, pp. 84–95.

[18] G. Gallo, M. D. Grigoriadis, and R. E. Tarjan, "A fast parametric maximum flow algorithm and applications," *SIAM Journal on Computing*, vol. 18, no. 1, pp. 30–55, 1989. [Online]. Available: https://doi.org/10.1137/0218003

[19] Y. Tang, Y. Shi, and X. Xiao, "Influence maximization in near-linear time: A martingale approach," in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 1539–1554. [Online]. Available: https://doi.org/10.1145/2723372.2723734

[20] H. T. Nguyen, M. T. Thai, and T. N. Dinh, "Stop-and-stare: Optimal sampling algorithms for viral marketing in billion-scale networks," in *Proceedings of the 2016 International Conference on Management of Data*, ser. SIGMOD '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 695–710.

[21] J. Tang, X. Tang, X. Xiao, and J. Yuan, "Online processing algorithms for influence maximization," in *Proceedings of the 2018 International Conference on Management of Data*, ser. SIGMOD '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 991–1005.

[22] Q. Guo, S. Wang, Z. Wei, and M. Chen, "Influence maximization revisited: Efficient reverse reachable set generation with bound tightened," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 2167–2181.

[23] Q. Guo, S. Wang, Z. Wei, W. Lin, and J. Tang, "Influence maximization revisited: Efficient sampling with bound tightened," *ACM Trans. Database Syst.*, vol. 47, no. 3, Aug. 2022.

[24] A. Goyal, W. Lu, and L. V. Lakshmanan, "Celf++: optimizing the greedy algorithm for influence maximization in social networks," in *Proceedings of the 20th International Conference Companion on World Wide Web*, ser. WWW '11. New York, NY, USA: Association for Computing Machinery, 2011, p. 47–48. [Online]. Available: https://doi.org/10.1145/1963192.1963217

[25] N. Barbieri, F. Bonchi, and G. Manco, "Topic-aware social influence propagation models," *Knowledge and information systems*, vol. 37, no. 3, pp. 555–584, 2013.

[26] S. Chen, J. Fan, G. Li, J. Feng, K.-l. Tan, and J. Tang, "Online topic-aware influence maximization," *Proc. VLDB Endow.*, vol. 8, no. 6, p. 666–677, Feb. 2015. [Online]. Available: https://doi.org/10.14778/2735703.2735706

[27] J. Liu, Z. Liang, H. Du, W. Xu, and X. Jia, "Topicrrc: a reverse sampling algorithm for maximizing online topic-aware rumor containment," *IEEE Transactions on Mobile Computing*, p. 1–15, 2026.

[28] J. Wang, Y. Wu, X. Wang, Y. Zhang, W. Zhang, and L. Qin, "Time-critical influence minimization via node blocking," *Proc. ACM Manag. Data*, vol. 3, no. 6, Dec. 2025. [Online]. Available: https://doi.org/10.1145/3769834

[29] W. Chen, W. Lu, and N. Zhang, "Time-critical influence maximization in social networks with time-delayed diffusion process," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 26, no. 1, 2012, pp. 591–598.

[30] X. Chang, J. Fu, Q. Liu, Y. Gao, and B. Zheng, "Time-aware influence minimization via blocking social networks," in *2025 IEEE 41st International Conference on Data Engineering (ICDE)*, 2025, pp. 557–570.

[31] M. Cerulli, C. D'Ambrosio, and A. Raiconi, "Time-aware influence propagation on networks: A survey," in *Emerging Technologies in Computational Sciences for Industry, Sustainability and Innovation*, M. Giacomini, S. Perotto, and G. Rozza, Eds. Cham: Springer Nature Switzerland, 2025, pp. 103–123.

[32] T. Cai, J. Li, A. Mian, R.-H. Li, T. Sellis, and J. X. Yu, "Target-aware holistic influence maximization in spatial social networks," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 4, pp. 1993–2007, 2022.

[33] G. Li, S. Chen, J. Feng, K.-l. Tan, and W.-s. Li, "Efficient location-aware influence maximization," in *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '14. New York, NY, USA: Association for Computing Machinery, 2014, p. 87–98. [Online]. Available: https://doi.org/10.1145/2588555.2588561

[34] T. Zhou, J. Cao, B. Liu, S. Xu, Z. Zhu, and J. Luo, "Location-based influence maximization in social networks," in *Proceedings of the 24th ACM International on Conference on Information and Knowledge Management*, ser. CIKM '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 1211–1220. [Online]. Available: https://doi.org/10.1145/2806416.2806462

[35] J. Zhu, S. Ghosh, and W. Wu, "Group influence maximization problem in social networks," *IEEE Transactions on Computational Social Systems*, vol. 6, no. 6, pp. 1156–1164, 2019.

[36] M. Zhou, W. Cao, H. Liao, and R. Mao, "Motif-oriented influence maximization for viral marketing in large-scale social networks," in *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, Eds., vol. 37. Curran Associates, Inc., 2024, pp. 135 861–135 882. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/file/f50b83e4ca6d57a9b59a2316ba298002-Paper-Conference.pdf

[37] T. Gao and K. Tajima, "Revenue maximization in campaigns with a viral-driven presale and a conformity-driven sales phase," *World Wide Web*, vol. 28, no. 6, pp. 1–32, 2025.

[38] T. Gao and T. Ito, "Beyond node influence: Activating nested subgraphs in propagation networks," *ACM Trans. Knowl. Discov. Data*, Jul. 2026, just Accepted. [Online]. Available: https://doi.org/10.1145/3828169

[39] A. Tsang, B. Wilder, E. Rice, M. Tambe, and Y. Zick, "Group-fairness in influence maximization," in *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI-19*. International Joint Conferences on Artificial Intelligence Organization, 7 2019, pp. 5997–6005. [Online]. Available: https://doi.org/10.24963/ijcai.2019/831

[40] A. Rahmattalabi, S. Jabbari, H. Lakkaraju, P. Vayanos, M. Izenberg, R. Brown, E. Rice, and M. Tambe, "Fair influence maximization: a welfare optimization approach," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 13, pp. 11 630–11 638, May 2021. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/17383

[41] X. Rui, Z. Wang, J. Zhao, L. Sun, and W. Chen, "Scalable fair influence maximization," in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS '23. Red Hook, NY, USA: Curran Associates Inc., 2023.

[42] Y. Feng, A. Patel, B. Cautis, and H. Vahabi, "Influence maximization with fairness at scale," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ser. KDD '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 4046–4055. [Online]. Available: https://doi.org/10.1145/3580305.3599847

[43] T. Gao and T. Ito, "Robust fair influence maximization under multiple community partitions," *Proc. ACM Manag. Data*, vol. 4, no. 1, Apr. 2026. [Online]. Available: https://doi.org/10.1145/3786692

[44] Y. Mehmood, F. Bonchi, and D. García-Soriano, "Spheres of influence for more effective viral marketing," in *Proceedings of the 2016 International Conference on Management of Data*, ser. SIGMOD '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 711–726. [Online]. Available: https://doi.org/10.1145/2882903.2915250

[45] Z. Wang, Y. Yang, J. Pei, L. Chu, and E. Chen, "Activity maximization by effective information diffusion in social networks," *IEEE Transactions on Knowledge and Data Engineering*, vol. 29, no. 11, pp. 2374–2387, 2017.

[46] Z. Hu, W. Zheng, and X. Lian, "Triangular stability maximization by influence spread over social networks," *Proc. VLDB Endow.*, vol. 16, no. 11, p. 2818–2831, Jul. 2023. [Online]. Available: https://doi.org/10.14778/3611479.3611490

[47] Y. Fang, X. Huang, L. Qin, Y. Zhang, W. Zhang, R. Cheng, and X. Lin, "A survey of community search over big graphs," *The VLDB Journal*, vol. 29, no. 1, pp. 353–392, 2020.

[48] Q. Liu, M. Zhao, X. Huang, J. Xu, and Y. Gao, "Truss-based community search over large directed graphs," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 2183–2197. [Online]. Available: https://doi.org/10.1145/3318464.3380587

[49] L. Chen, C. Liu, R. Zhou, J. Li, X. Yang, and B. Wang, "Maximum co-located community search in large scale social networks," *Proc. VLDB Endow.*, vol. 11, no. 10, p. 1233–1246, Jun. 2018. [Online]. Available: https://doi.org/10.14778/3231751.3231755

[50] X. Huang and L. V. S. Lakshmanan, "Attribute-driven community search," *Proc. VLDB Endow.*, vol. 10, no. 9, p. 949–960, May 2017. [Online]. Available: https://doi.org/10.14778/3099622.3099626

[51] Y. Jiang, Y. Fang, C. Ma, X. Cao, and C. Li, "Effective community search over large star-schema heterogeneous information networks," *Proc. VLDB Endow.*, vol. 15, no. 11, p. 2307–2320, Jul. 2022. [Online]. Available: https://doi.org/10.14778/3551793.3551795

[52] Y. Zhou, Y. Fang, W. Luo, and Y. Ye, "Influential community search over large heterogeneous information networks," *Proc. VLDB Endow.*, vol. 16, no. 8, p. 2047–2060, Apr. 2023. [Online]. Available: https://doi.org/10.14778/3594512.3594532

[53] S. Chen, R. Wei, D. Popova, and A. Thomo, "Efficient computation of importance based communities in web-scale networks using a single machine," in *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management*, ser. CIKM '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 1553–1562. [Online]. Available: https://doi.org/10.1145/2983323.2983836

[54] W. Luo, X. Zhou, J. Yang, P. Peng, G. Xiao, and Y. Gao, "Efficient approaches to top-r influential community search," *IEEE Internet of Things Journal*, vol. 8, no. 16, pp. 12 650–12 657, 2021.

[55] R.-H. Li, L. Qin, J. X. Yu, and R. Mao, "Finding influential communities in massive networks," *The VLDB Journal*, vol. 26, no. 6, pp. 751–776, 2017.

[56] ——, "Influential community search in large networks," *Proc. VLDB Endow.*, vol. 8, no. 5, p. 509–520, Jan. 2015. [Online]. Available: https://doi.org/10.14778/2735479.2735484

[57] Y. Sun, Y. Zhou, Y. Fang, C. Chen, Y. Hu, and Y. Hu, "Efficient influential community search over dynamic graphs," *Proc. ACM Manag. Data*, vol. 4, no. 1, Apr. 2026. [Online]. Available: https://doi.org/10.1145/3786644

[58] Z. Lu, Y. Zhu, W. Li, W. Wu, and X. Cheng, "Influence-based community partition for social networks," *Computational Social Networks*, vol. 1, no. 1, p. 1, 2014.

[59] Q. Ni, J. Guo, W. Wu, and H. Wang, "Influence-based community partition with sandwich method for social networks," *IEEE Transactions on Computational Social Systems*, vol. 10, no. 2, pp. 819–830, 2023.

[60] Q. Ni, J. Guo, W. Wu, H. Wang, and J. Wu, "Continuous influence-based community partition for social networks," *IEEE Transactions on Network Science and Engineering*, vol. 9, no. 3, pp. 1187–1197, 2022.

[61] X. Chang, Q. Liu, Y. Gao, B. Zheng, Y. Cai, and Q. Li, "The most influenced community search on social networks," in *2025 IEEE 41st International Conference on Data Engineering (ICDE)*, 2025, pp. 2601–2614.

[62] C. Giatsidis, D. M. Thilikos, and M. Vazirgiannis, "D-cores: measuring collaboration of directed graphs based on degeneracy," *Knowledge and information systems*, vol. 35, no. 2, pp. 311–343, 2013.

[63] T. Lanciano, A. Miyauchi, A. Fazzone, and F. Bonchi, "A survey on the densest subgraph problem and its variants," *ACM Comput. Surv.*, vol. 56, no. 8, Apr. 2024. [Online]. Available: https://doi.org/10.1145/3653298

[64] C. Chekuri, K. Quanrud, and M. R. Torres, *Densest Subgraph: Supermodularity, Iterative Peeling, and Flow*, 2022, pp. 1531–1555. [Online]. Available: https://epubs.siam.org/doi/abs/10.1137/1.9781611977073.64

[65] Y. Dai, M. Qiao, and L. Chang, "Anchored densest subgraph," in *Proceedings of the 2022 International Conference on Management of Data*, ser. SIGMOD '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 1200–1213. [Online]. Available: https://doi.org/10.1145/3514221.3517890

[66] M. Sozio and A. Gionis, "The community-search problem and how to plan a successful cocktail party," in *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '10. New York, NY, USA: Association for Computing Machinery, 2010, p. 939–948. [Online]. Available: https://doi.org/10.1145/1835804.1835923

[67] A. Fazzone, T. Lanciano, R. Denni, C. E. Tsourakakis, and F. Bonchi, "Discovering polarization niches via dense subgraphs with attractors and repulsers," *Proc. VLDB Endow.*, vol. 15, no. 13, p. 3883–3896, Sep. 2022. [Online]. Available: https://doi.org/10.14778/3565838.3565843

[68] F. Bonchi, D. García-Soriano, A. Miyauchi, and C. E. Tsourakakis, "Finding densest k-connected subgraphs," *Discrete Applied Mathematics*, vol. 305, pp. 34–47, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0166218X21003395

[69] C. Tsourakakis, "The k-clique densest subgraph problem," in *Proceedings of the 24th International Conference on World Wide Web*, ser. WWW '15. Republic and Canton of Geneva, CHE: International World Wide Web Conferences Steering Committee, 2015, p. 1122–1132. [Online]. Available: https://doi.org/10.1145/2736277.2741098

[70] Y. Zhou, L. Liang, and Y. Fang, "Efficient and scalable directed densest subgraph discovery," *Proc. ACM Manag. Data*, vol. 3, no. 6, Dec. 2025. [Online]. Available: https://doi.org/10.1145/3769785

[71] X. Ye, R.-H. Li, L. Liang, Z. Liu, L. Lin, and G. Wang, "Efficient and effective anchored densest subgraph search: A convex-programming

based approach," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ser. KDD '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 3907–3918. [Online]. Available: https://doi.org/10.1145/3637528.3671727
[72] Z. Zou, "Polynomial-time algorithm for finding densest subgraphs in uncertain graphs," in *Proceedings of MLG Workshop*, 2013.
[73] A. Saha, X. Ke, A. Khan, and C. Long, "Most probable densest subgraphs," in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*, 2023, pp. 1447–1460.
[74] A. Miyauchi and A. Takeda, "Robust densest subgraph discovery," in *2018 IEEE International Conference on Data Mining (ICDM)*, 2018, pp. 1188–1193.
[75] Y. Kuroki, A. Miyauchi, J. Honda, and M. Sugiyama, "Online dense subgraph discovery via blurred-graph feedback," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, H. D. III and A. Singh, Eds., vol. 119. PMLR, 13–18 Jul 2020, pp. 5522–5532. [Online]. Available: https://proceedings.mlr.press/v119/kuroki20a.html
[76] F. Bonchi, F. Gullo, A. Kaltenbrunner, and Y. Volkovich, "Core decomposition of uncertain graphs," in *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '14. New York, NY, USA: Association for Computing Machinery, 2014, p. 1316–1325. [Online]. Available: https://doi.org/10.1145/2623330.2623655
[77] M. Granovetter, "Threshold models of collective behavior," *American journal of sociology*, vol. 83, no. 6, pp. 1420–1443, 1978.
[78] T. C. Schelling, *Micromotives and macrobehavior*. WW Norton & Company, 2006.
[79] J. Goldenberg, B. Libai, and E. Muller, "Talk of the network: A complex systems look at the underlying process of word-of-mouth," *Marketing letters*, vol. 12, no. 3, pp. 211–223, 2001.
[80] ——, "Using complex systems analysis to advance marketing theory development: Modeling heterogeneity effects on new product growth through stochastic cellular automata," *Academy of Marketing Science Review*, vol. 9, no. 3, pp. 1–18, 2001.
[81] J. Gill, "Computational complexity of probabilistic turing machines," *SIAM Journal on Computing*, vol. 6, no. 4, pp. 675–695, 1977. [Online]. Available: https://doi.org/10.1137/0206049
[82] J. Goldsmith, M. Hagen, and M. Mundhenk, "Complexity of dnf minimization and isomorphism testing for monotone formulas," *Information and Computation*, vol. 206, no. 6, pp. 760–775, 2008.
[83] K. Zhu, W. Zhang, G. Zhu, Y. Zhang, and X. Lin, "Bmc: An efficient method to evaluate probabilistic reachability queries," in *Database Systems for Advanced Applications*, J. X. Yu, M. H. Kim, and R. Unland, Eds. Berlin, Heidelberg: Springer, 2011, pp. 434–449.
[84] X. Ke, A. Khan, and L. L. H. Quan, "An in-depth comparison of s-t reliability algorithms over uncertain graphs," *Proc. VLDB Endow.*, vol. 12, no. 8, p. 864–876, Apr. 2019. [Online]. Available: https://doi.org/10.14778/3324301.3324304
[85] L. G. Valiant, "The complexity of enumeration and reliability problems," *SIAM Journal on Computing*, vol. 8, no. 3, pp. 410–421, 1979. [Online]. Available: https://doi.org/10.1137/0208032
[86] J. Tang, K. Huang, X. Xiao, L. V. Lakshmanan, X. Tang, A. Sun, and A. Lim, "Efficient approximation algorithms for adaptive seed minimization," in *Proceedings of the 2019 International Conference on Management of Data*, ser. SIGMOD '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 1096–1113. [Online]. Available: https://doi.org/10.1145/3299869.3319881
[87] Y. Zhu, J. Tang, X. Tang, S. Wang, and A. Lim, "2-hop+ sampling: Efficient and effective influence estimation," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 2, pp. 1088–1103, 2023.
[88] H. T. Nguyen, T. P. Nguyen, T. N. Vu, and T. N. Dinh, "Outward influence and cascade size estimation in billion-scale networks," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 1, no. 1, Jun. 2017. [Online]. Available: https://doi.org/10.1145/3084457
[89] S. Khuller and B. Saha, "On finding dense subgraphs," in *Automata, Languages and Programming*, S. Albers, A. Marchetti-Spaccamela, Y. Matias, S. Nikoletseas, and W. Thomas, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 597–608.
[90] J. B. Orlin, "Max flows in o(nm) time, or better," in *Proceedings of the Forty-Fifth Annual ACM Symposium on Theory of Computing*, ser. STOC '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 765–774. [Online]. Available: https://doi.org/10.1145/2488608.2488705
[91] R. A. Rossi and N. K. Ahmed, "The network data repository with interactive graph analytics and visualization," in *AAAI*, 2015. [Online]. Available: http://networkrepository.com
[92] J. Leskovec and A. Krevl, "SNAP Datasets: Stanford large network dataset collection," http://snap.stanford.edu/data, Jun. 2014.
[93] C. Tsourakakis, F. Bonchi, A. Gionis, F. Gullo, and M. Tsiarli, "Denser than the densest subgraph: extracting optimal quasi-cliques with quality guarantees," in *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 104–112. [Online]. Available: https://doi.org/10.1145/2487575.2487645
[94] P. Lee and L. V. Lakshmanan, "Query-driven maximum quasi-clique search," in *Proceedings of the 2016 SIAM International Conference on Data Mining (SDM)*. SIAM, 2016, pp. 522–530.
[95] Y. Tang, X. Xiao, and Y. Shi, "Influence maximization: near-optimal time complexity meets practical efficiency," in *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '14. New York, NY, USA: Association for Computing Machinery, 2014, p. 75–86. [Online]. Available: https://doi.org/10.1145/2588555.2593670
[96] W. Hoeffding, "Probability inequalities for sums of bounded random variables," *Journal of the American Statistical Association*, vol. 58, no. 301, pp. 13–30, 1963.
[97] J. Leskovec, A. Krause, C. Guestrin, C. Faloutsos, J. VanBriesen, and N. Glance, "Cost-effective outbreak detection in networks," in *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '07. New York, NY, USA: Association for Computing Machinery, 2007, p. 420–429. [Online]. Available: https://doi.org/10.1145/1281192.1281239

APPENDIX

This appendix collects the proofs omitted from the main text. For clarity, the material is organized following the structure of Section VI.

The proofs have the following dependency chain.

- Lemma 6, Lemma 16, and Lemma 17 $\Rightarrow$ Theorem 7.
- Lemma 19 and Lemma 20 $\Rightarrow$ Lemma 10 $\Rightarrow$ Theorem 11.
- Lemma 5, Theorem 7, and Theorem 11 $\Rightarrow$ Theorem 12.

A. Proofs for the Pilot Stage

We first provide the proof of the coverage guarantee for the pilot stage. The key idea is that any pair with co-activation similarity at least τ is observed in each pilot realization with probability at least τ, so the probability that it is missed throughout all T_0 pilot samples decays exponentially in T_0.

Proof of Lemma 5. Fix any unordered pair $\{u, v\} \subseteq V \setminus S$ with $W_{uv} \geq \tau$. For each trial t, the event $\{u \in \mathcal{A}_t \wedge v \in \mathcal{A}_t\}$ occurs with probability W_{uv}. Hence, the probability that the pair is never observed in T_0 trials is

$$\Pr\left(\sum_{t=1}^{T_0} \mathbb{I}(u \in \mathcal{A}_t \wedge v \in \mathcal{A}_t) = 0\right) = (1-W_{uv})^{T_0} \leq (1-\tau)^{T_0}.$$

By a union bound over at most $\binom{n_0}{2}$ unordered pairs in $V \setminus S$,

$$\Pr\left(\exists \{u, v\} \subseteq V \setminus S : W_{uv} \geq \tau, \{u, v\} \notin \widehat{E}^{\tau}\right) \leq \binom{n_0}{2}(1-\tau)^{T_0}.$$

By the assumed lower bound on T_0, the right-hand side is at most δ_0, completing the proof. □

B. Auxiliary Results for the Certified Stage

We next collect the auxiliary results used in the analysis of the certified stage. We begin with the proof of Lemma 6, which establishes that the checkpoint-wise confidence bounds are simultaneously valid for all tracked pairs and all checkpoint times. The proof combines Hoeffding's inequality with a union bound over the tracked pairs and the geometric checkpoint schedule.

Proof of Lemma 6. Fix $e = \{u, v\} \in \widehat{E}^\tau$ and a checkpoint $t \in \mathcal{T}$. Let $\widehat{W}_e(t) := C_e/t$, where C_e is the counter in Algorithm 3 (line 10). By Hoeffding's inequality for i.i.d. Bernoulli samples [96],

$$\Pr\Big(\big|\widehat{W}_e(t) - W_e\big| \geq r(t)\Big) \leq 2\exp\big(-2t \cdot r(t)^2\big). \tag{14}$$

For $t \in \mathcal{T}$, substituting $r(t)$ into the right-hand side of Eq. (14) yields

$$\Pr\Big(\big|\widehat{W}_e(t) - W_e\big| \geq r(t)\Big) \leq \frac{2\delta_1}{3|\widehat{E}^\tau|(i+1)(i+2)}.$$

By a union bound over all $e \in \widehat{E}^\tau$ and all checkpoints $t \in \mathcal{T}$,

$$\Pr\Big(\exists e \in \widehat{E}^\tau, t \in \mathcal{T} : \big|\widehat{W}_e(t) - W_e\big| \geq r(t)\Big)$$
$$\leq \sum_{e\in\widehat{E}^\tau}\sum_{i\geq 0} \frac{2\delta_1}{3|\widehat{E}^\tau|(i+1)(i+2)} = \frac{2\delta_1}{3}\sum_{i\geq 0}\frac{1}{(i+1)(i+2)} \leq \frac{2\delta_1}{3},$$

where we use $\sum_{i\geq 0}\frac{1}{(i+1)(i+2)} = 1$. Therefore, with probability at least $1 - \frac{2\delta_1}{3}$, we have $\big|\widehat{W}_e(t) - W_e\big| \leq r(t)$ for all $e \in \widehat{E}^\tau$ and all $t \in \mathcal{T}$.

Condition on the event that $\big|\widehat{W}_e(t) - W_e\big| \leq r(t)$ holds for all $e \in \widehat{E}^\tau$ and all $t \in \mathcal{T}$. Fix any e and $t \in \mathcal{T}$.

If $W_e < \tau$, then $W_e \cdot \mathbb{I}[W_e \geq \tau] = 0$. Moreover, $\widehat{W}_e(t) - r(t) \leq W_e < \tau$, so the indicator in line 15 is zero and thus $W_e^-(t) = 0$. Since $W_e^+(t) \geq 0$ by construction, Eq. (6) holds.

If $W_e \geq \tau$, then $\widehat{W}_e(t) + r(t) \geq W_e \geq \tau$, so the indicator in line 16 equals one. For the lower bound, if $\widehat{W}_e(t) - r(t) < \tau$, then line 15 gives $W_e^-(t) = 0 \leq W_e$. Otherwise, the indicator in line 15 equals one, and hence $W_e^-(t) = \widehat{W}_e(t) - r(t) \leq W_e$. Thus, $W_e^-(t) \leq W_e$ in either case. Moreover, since $W_e \leq 1$, line 16 gives $W_e^+(t) = \min\{1, \widehat{W}_e(t) + r(t)\} \geq W_e$. Hence, Eq. (6) holds in this case as well. □

We first record a monotonicity property of the density objective with respect to edge weights. This fact will be used to compare the optimistic and pessimistic instances induced by the confidence bounds.

Lemma 14. *Fix $\widehat{E}^\tau \subseteq \binom{V\setminus S}{2}$. Let A and B be two nonnegative weight functions on $\widehat{E}^\tau$ such that $A_e \leq B_e$ for all $e \in \widehat{E}^\tau$. Then for every nonempty $U \subseteq V \setminus S$,*

$$\rho_A(U) \leq \rho_B(U), \tag{15}$$

where $\rho_A(\cdot)$ and $\rho_B(\cdot)$ are defined in Section III-C.

Proof. For any nonempty $U \subseteq V \setminus S$, the condition $A_e \leq B_e$ for all $e \in \widehat{E}^\tau$ implies

$$\sum_{e\in\widehat{E}^\tau(U)} A_e \leq \sum_{e\in\widehat{E}^\tau(U)} B_e.$$

Dividing both sides by $|U|$ yields Eq. (15). □

We next record the standard approximation guarantee of weighted peeling for the weighted densest-subgraph problem. This result will be used to relate the peeling output in Algorithm 2 to the corresponding optimum.

Lemma 15. *Fix $\widehat{E}^\tau \subseteq \binom{V\setminus S}{2}$ and let W' be any nonnegative weight function on $\widehat{E}^\tau$. Let $U_{\max}$ be the node set returned by Algorithm 2 on input $(V, S, \widehat{E}^\tau, W')$. Then*

$$\rho_{W'}(U_{\max}) \geq \frac{1}{2}\max_{\emptyset\neq U\subseteq V\setminus S}\rho_{W'}(U). \tag{16}$$

Proof. Let $U^* \in \arg\max_{\emptyset\neq U\subseteq V\setminus S}\rho_{W'}(U)$ and $\rho^* := \rho_{W'}(U^*)$. We first show that U^* contains a nonempty subset $H \subseteq U^*$ such that every node in H has weighted degree at least ρ^* within the subgraph induced by H.

Starting from U^*, repeatedly remove any node whose current weighted degree within the current set is strictly smaller than ρ^*, and stop when no such node exists. If all nodes were removed, then at each removal step the total induced weight would decrease by strictly less than ρ^*. Summing over all removed nodes would therefore yield $W'(U^*) < \rho^*|U^*|$, which contradicts the definition $\rho^* = W'(U^*)/|U^*|$. Hence the remaining set H is nonempty, and every node in H has weighted degree at least ρ^* within H.

Now consider Algorithm 2 run on $(V, S, \widehat{E}^\tau, W')$. Let U be the current remaining set immediately before the first node of H is removed. Then $H \subseteq U$, and each node in H has weighted degree at least ρ^* within H, hence also at least ρ^* within U. Since weighted peeling removes a node of minimum current weighted degree, the minimum weighted degree in U is at least ρ^*.

The sum of weighted degrees in U equals $2W'(U)$, so the average weighted degree in U is

$$\frac{2W'(U)}{|U|} = 2\rho_{W'}(U).$$

Since the minimum weighted degree is at most the average weighted degree, we obtain $2\rho_{W'}(U) \geq \rho^*$. Algorithm 2 returns the intermediate set with the maximum density over all iterations, so

$$\rho_{W'}(U_{\max}) \geq \rho_{W'}(U) \geq \frac{\rho^*}{2}.$$

This proves Eq. (16). □

We next justify the optimistic and pessimistic bounds used in the stopping rule. To avoid ambiguity, we introduce an auxiliary thresholded weight function $\bar{W}^\tau$ on the tracked edge set $\widehat{E}^\tau$, defined by

$$\bar{W}_e^\tau := W_e \cdot \mathbb{I}[W_e \geq \tau].$$

Under the event $E^\tau \subseteq \widehat{E}^\tau$, every pair $\{u, v\}$ with $W_{uv} \geq \tau$ is contained in $\widehat{E}^\tau$, and every pair in $\widehat{E}^\tau \setminus E^\tau$ has auxiliary weight $\bar{W}_e^\tau = 0$. Hence, for every nonempty $U \subseteq V \setminus S$,

$$\sum_{e \in \widehat{E}^\tau(U)} \bar{W}_e^\tau = \sum_{e \in E^\tau(U)} W_e = W^\tau(U),$$

which implies

$$\rho_{\bar{W}^\tau}(U) = \rho(U). \tag{17}$$

We first justify the optimistic side of the stopping rule. Since $W^+(t)$ upper-bounds the thresholded edge weights, the weighted densest-subgraph instance induced by $W^+(t)$ should provide an upper estimate of the true optimum. The following lemma formalizes this claim for the peeling solution $U^+(t)$.

Lemma 16. *Fix a checkpoint $t \in \mathcal{T}$. Assume $E^\tau \subseteq \widehat{E}^\tau$ and $\bar{W}_e^\tau \leq W_e^+(t)$ for all $e \in \widehat{E}^\tau$. Let $U^+(t)$ be the set returned by Algorithm 2 on $(V, S, \widehat{E}^\tau, W^+(t))$. Then*

$$2\rho_{W^+(t)}\big(U^+(t)\big) \geq OPT. \tag{18}$$

Proof. By Lemma 15 applied to $W^+(t)$,

$$2\rho_{W^+(t)}\big(U^+(t)\big) \geq \max_{\emptyset \neq U \subseteq V \setminus S} \rho_{W^+(t)}(U). \tag{19}$$

Since $\bar{W}_e^\tau \leq W_e^+(t)$ for all $e \in \widehat{E}^\tau$, Lemma 14 implies that for every nonempty $U \subseteq V \setminus S$, $\rho_{W^+(t)}(U) \geq \rho_{\bar{W}^\tau}(U)$. In particular, for the optimal set U° defined in Section III-C,

$$\max_{\emptyset \neq U \subseteq V \setminus S} \rho_{W^+(t)}(U) \geq \rho_{W^+(t)}(U^\circ) \geq \rho_{\bar{W}^\tau}(U^\circ) = \rho(U^\circ) = OPT,$$

where the first equality follows from Eq. (17). Combining this inequality with Eq. (19) yields Eq. (18). □

We next justify the pessimistic side of the stopping rule. Since $W^-(t)$ lower-bounds the thresholded edge weights, evaluating any node set under $W^-(t)$ should never overestimate its true objective value. The following lemma makes this lower-bound property precise.

Lemma 17. *Fix a checkpoint $t \in \mathcal{T}$. Assume $E^\tau \subseteq \widehat{E}^\tau$ and $W_e^-(t) \leq \bar{W}_e^\tau$ for all $e \in \widehat{E}^\tau$. Then for every nonempty $U \subseteq V \setminus S$,*

$$\rho_{W^-(t)}(U) \leq \rho(U). \tag{20}$$

Proof. By assumption, $W_e^-(t) \leq \bar{W}_e^\tau$ for all $e \in \widehat{E}^\tau$. Therefore, Lemma 14 implies that for every nonempty $U \subseteq V \setminus S$, $\rho_{W^-(t)}(U) \leq \rho_{\bar{W}^\tau}(U)$. By Eq. (17), we have $\rho_{W^-(t)}(U) \leq \rho(U)$, which proves Eq. (20). □

We are now ready to prove the correctness of the early-stopping rule. The key idea is to combine the optimistic upper bound from Lemma 16 with the pessimistic lower bound from Lemma 17. Together with the stopping condition in Eq. (7), these two ingredients imply that once the rule is triggered, the current solution $\widehat{U}(t)$ already achieves the desired approximation guarantee.

Proof of Theorem 7. Combining Eq. (18) in Lemma 16 with Eq. (7) yields

$$OPT \leq (2 + \varepsilon) \cdot \rho_{W^-(t)}\big(\widehat{U}(t)\big). \tag{21}$$

By Eq. (20) in Lemma 17, we have $\rho_{W^-(t)}(\widehat{U}(t)) \leq \rho(\widehat{U}(t))$. Substituting this inequality into Eq. (21) gives Eq. (8). □

Therefore, whenever the stopping rule holds at a checkpoint t, the certified stage may terminate immediately and return $\widehat{U}(t)$ while preserving the claimed $(2 + \varepsilon)$-approximation guarantee.

C. Details for Justifying $T_{\max}$

Before proceeding, we further elaborate on the role of Assumption 8. As explained in Remark 9 in the main text, this assumption is only needed for the worst-case fallback guarantee at $t = T_{\max}$. The following remark gives a more detailed explanation of why the assumption mainly excludes a threshold-critical case on the tracked set and why a small perturbation of τ typically avoids this issue.

Remark 18 (Further details on Remark 9). We provide a more detailed justification of Remark 9. Assumption 8 fails only when the threshold τ is critical on the tracked set, i.e., when $\tau = W_e$ for some $e \in \widehat{E}^\tau$. Let $\mathcal{W} \coloneqq \{W_e : e \in \widehat{E}^\tau\}$, and define

$$w^- \coloneqq \max\big(\{w \in \mathcal{W} : w < \tau\} \cup \{0\}\big).$$

Since $\mathcal{W}$ is finite, one may choose a perturbed threshold $\tau' \in (w^-, \tau)$ such that $\tau' \notin \mathcal{W}$, e.g., $\tau' = (w^- + \tau)/2$. By construction, no tracked weight lies in the interval $[\tau', \tau)$, i.e., $W_e \notin [\tau', \tau)$ for all $e \in \widehat{E}^\tau$. Hence, thresholding on the tracked set remains unchanged, in the sense that $\mathbb{I}[W_e \geq \tau'] = \mathbb{I}[W_e \geq \tau]$ for all $e \in \widehat{E}^\tau$. At the same time, a positive margin now exists at τ', namely

$$\gamma' \coloneqq \min_{e \in \widehat{E}^\tau} |W_e - \tau'| > 0.$$

This explains why an arbitrarily small perturbation of τ typically avoids the critical case without changing the thresholded structure on $\widehat{E}^\tau$. However, computing γ or γ' exactly is intractable in general, since it requires evaluating the exact co-activation probabilities W_e, which is #P-hard under the diffusion models considered in this paper.

With this interpretation in mind, we now return to the analysis at $t = T_{\max}$. The next lemma establishes a uniform concentration bound for the empirical similarities at the final time. This bound will later be combined with Assumption 8 to rule out threshold inversions.

Lemma 19. *With probability at least $1 - \delta_1/3$, the event in Eq.* (10) *holds simultaneously for all $e \in \widehat{E}^\tau$.*

Proof. Fix any $e \in \widehat{E}^\tau$. By Hoeffding's inequality [96],

$$\Pr\Big(\big|\widehat{W}_e - W_e\big| \geq r\Big) \leq 2\exp(-2T_{\max}r^2). \tag{22}$$

Substituting Eq. (9) into Eq. (22) yields

$$\Pr\Big(\big|\widehat{W}_e - W_e\big| \geq r\Big) \leq \frac{\delta_1}{3|\widehat{E}^\tau|}.$$

Applying a union bound over all $e \in \widehat{E}^\tau$ proves the claim. □

Lemma 19 shows that, at the final time $t = T_{\max}$, all empirical similarities are uniformly close to their true values with high probability. However, this concentration bound alone does not guarantee that thresholding at level τ is preserved. The following lemma shows that, under Assumption 8, no threshold inversion can occur when $rl\gamma$.

Lemma 20. *Assume that Assumption 8 holds. On the event in Eq.* (10)*, if $r \le \gamma$, then*

$$\mathbb{I}\Big[\widehat{W}_e \ge \tau\Big] = \mathbb{I}[W_e \ge \tau]$$

holds for all $e \in \widehat{E}^\tau$.

Proof. Fix any $e \in \widehat{E}^\tau$. By Assumption 8, either $W_e \ge \tau + \gamma$ or $W_e \le \tau - \gamma$.

If $W_e \ge \tau + \gamma$, then on the event in Eq. (10),

$$\widehat{W}_e \ge W_e - r \ge \tau + \gamma - r \ge \tau,$$

so $\mathbb{I}[\widehat{W}_e \ge \tau] = 1 = \mathbb{I}[W_e \ge \tau]$.

If $W_e \le \tau - \gamma$, then on the event in Eq. (10),

$$\widehat{W}_e \le W_e + r \le \tau - \gamma + r \le \tau,$$

so $\mathbb{I}[\widehat{W}_e \ge \tau] = 0 = \mathbb{I}[W_e \ge \tau]$.

This proves the claim. □

We now prove Lemma 10. The key step is to lift the edge-wise concentration bound to a uniform additive error bound on the density objective. Under Assumption 8, Lemma 20 ensures that the thresholding decisions at $t = T_{\max}$ agree with the true ones, so the remaining error comes only from the magnitudes of the retained edge weights.

Proof of Lemma 10. Fix any nonempty $U \subseteq V \setminus S$. If $|U| = 1$, then both sides equal 0 and Eq. (11) holds.

Assume $|U| \ge 2$. By Lemma 20, the thresholding in line 24 matches the true thresholding on $\widehat{E}^\tau$. Hence, for all $e \in \widehat{E}^\tau$, we have

$$\big|\widetilde{W}_e(T_{\max}) - \bar{W}_e^\tau\big| \le r.$$

Summing over the pairs induced by U and dividing by $|U|$ yields

$$\left|\rho_{\widetilde{W}(T_{\max})}(U) - \frac{\sum_{e \in \widehat{E}^\tau(U)} \bar{W}_e^\tau}{|U|}\right| \le \frac{r}{|U|} \cdot |\widehat{E}^\tau(U)|. \quad (23)$$

Moreover, $|\widehat{E}^\tau(U)| \le \binom{|U|}{2}$ implies $|\widehat{E}^\tau(U)|/|U| \le (|U| - 1)/2 \le (n_0 - 1)/2$, and $|\widehat{E}^\tau(U)| \le |\widehat{E}^\tau|$ implies $|\widehat{E}^\tau(U)|/|U| \le |\widehat{E}^\tau|/2$. Thus,

$$|\widehat{E}^\tau(U)|/|U| \le L/2. \quad (24)$$

Finally, under $E^\tau \subseteq \widehat{E}^\tau$, Eq. (17) implies that

$$\frac{\sum_{e \in \widehat{E}^\tau(U)} \bar{W}_e^\tau}{|U|} = \rho_{\bar{W}^\tau}(U) = \rho(U).$$

Combining this identity with Eqs. (23) and (24) yields Eq. (11). □

Lemma 10 shows that the final empirical weighted instance is uniformly close to the true thresholded instance in terms of the density objective. We now combine this additive control with the 2-approximation guarantee of weighted peeling. The proof proceeds in two steps. We first derive an additive approximation bound for the fallback solution at $t = T_{\max}$. We then convert this additive bound into the desired multiplicative guarantee.

Proof of Theorem 11. By Lemma 10, for every nonempty $U \subseteq V \setminus S$,

$$\rho(U) \le \rho_{\widetilde{W}(T_{\max})}(U) + \frac{r}{2}L.$$

Taking the maximum over all nonempty $U \subseteq V \setminus S$ yields

$$OPT \le \max_{\emptyset \ne U \subseteq V \setminus S} \rho_{\widetilde{W}(T_{\max})}(U) + \frac{r}{2}L. \quad (25)$$

On the other hand, by Lemma 15 applied to $\widetilde{W}(T_{\max})$, the peeling output $\widehat{U}$ satisfies

$$\rho_{\widetilde{W}(T_{\max})}(\widehat{U}) \ge \frac{1}{2} \max_{\emptyset \ne U \subseteq V \setminus S} \rho_{\widetilde{W}(T_{\max})}(U). \quad (26)$$

Applying Lemma 10 again with $U = \widehat{U}$ gives

$$\rho(\widehat{U}) \ge \rho_{\widetilde{W}(T_{\max})}(\widehat{U}) - \frac{r}{2}L. \quad (27)$$

Combining Eqs. (25), (26), and (27), we obtain

$$\rho(\widehat{U}) \ge \frac{1}{2}OPT - \frac{3r}{4}L. \quad (28)$$

It remains to convert the additive error term into a multiplicative one. Under the pilot success event $E^\tau \subseteq \widehat{E}^\tau$, if $E^\tau = \varnothing$, then $OPT = 0$, and Eq. (12) holds trivially. Otherwise, there exists some $\{u, v\} \in E^\tau$ with $u, v \in V \setminus S$. For $U = \{u, v\}$, we have $\rho(U) \ge \tau/2$, and therefore $OPT \ge \tau/2$.

Using the condition $\frac{r}{2}L \le \frac{\varepsilon\tau}{6(2+\varepsilon)}$, we obtain

$$\frac{3r}{4}L \le \frac{\varepsilon\tau}{4(2+\varepsilon)} \le \frac{\varepsilon}{2+\varepsilon} \cdot \frac{OPT}{2}. \quad (29)$$

Substituting Eq. (29) into Eq. (28) yields Eq. (12). □

Therefore, under Assumption 8, the fallback solution returned at $t = T_{\max}$ also satisfies the claimed $(2+\varepsilon)$-approximation guarantee.

D. Proof of the Overall Guarantee

We finally prove the overall guarantee of Algorithm 4. The argument combines the pilot-stage coverage guarantee, the checkpoint-wise confidence bounds for the certified stage, and the fallback guarantee at $t = T_{\max}$. A union bound is then used to combine the corresponding high-probability events.

Proof of Theorem 12. Algorithm 4 sets $\delta_0 = \delta/2$ and $\delta_1 = \delta/2$. By Lemma 5, $\Pr(E^\tau \subseteq \widehat{E}^\tau) \ge 1 - \delta_0$. By Lemma 6, with probability at least $1 - 2\delta_1/3$, the checkpoint confidence bounds in Eq. (6) hold simultaneously for all $e \in \widehat{E}^\tau$ and all $t \in \mathcal{T}$. By Lemma 19, with probability at least $1 - \delta_1/3$, the

final-time concentration event in Eq. (10) holds at $t = T_{\max}$. By a union bound, these three events hold simultaneously with probability at least $1 - \delta$.

Condition on this intersection of events. If Algorithm 3 terminates early at some checkpoint $t \in \mathcal{T}$, then Theorem 7 implies Eq. (13). If Algorithm 3 does not terminate early and returns $\widehat{U}$ in line 26, then under Assumption 8 and the choice of $T_{\max}$ in Eq. (4), Theorem 11 implies Eq. (13). □

This completes the proof of the overall approximation guarantee stated in Theorem 12.

Remark 21 (Counterexample for Remark 13). We show that without Assumption 8, the fallback solution returned at $t = T_{\max}$ may fail to satisfy any positive worst-case multiplicative approximation guarantee.

Consider an IC instance with seed set $S = \{s\}$ and candidate node set $V \setminus S = \{u, v\}$. Add two directed edges, namely $s \to u$ with activation probability 1, and $s \to v$ with activation probability τ. Then u is always activated, and v is activated with probability τ. Hence $W_{uv} = \Pr(u \in \mathcal{A}(S) \wedge v \in \mathcal{A}(S)) = \tau$. Therefore, $E^\tau = \{\{u, v\}\}$, and for $U = \{u, v\}$, $\rho(U) = \frac{W^\tau(U)}{|U|} = \frac{\tau}{2}$. Since this is the only nonempty set with positive density, we have $OPT = \frac{\tau}{2} > 0$.

Now condition on the pilot success event $E^\tau \subseteq \widehat{E}^\tau$, so that the pair $\{u, v\}$ is tracked by the certified stage. At time $t = T_{\max}$, let $\widehat{W}_{uv}$ be the empirical estimate of W_{uv}. The event that v is never activated in the $T_{\max}$ certified samples has probability $(1-\tau)^{T_{\max}} > 0$. On this event, we have $\widehat{W}_{uv} = 0 < \tau$, so line 24 sets $\widetilde{W}_{uv}(T_{\max}) = 0$. Consequently, the final empirical weighted instance contains no positive-weight edge, and the fallback output $\widehat{U}$ satisfies $\rho(\widehat{U}) = 0$.

Thus, with positive probability, the fallback solution at $t = T_{\max}$ has objective value 0, whereas $OPT = \tau/2 > 0$. In particular, no positive worst-case multiplicative approximation guarantee can be ensured without Assumption 8.

This section provides several low-level implementation details to complement the experimental setup in the main text. The goal is to clarify the data representations used by the proposed methods and the heuristic baselines, and to show that the implementation-level comparison is conducted under consistent storage choices.

For the proposed algorithm, the certified stage stores the counters of tracked node pairs using an STL container of type `vector`. Since each tracked pair in $\widehat{E}^\tau$ is assigned a unique index, a single vector of length $|\widehat{E}^\tau|$ is sufficient to maintain the corresponding pairwise counters and estimates.

For the heuristic baselines, the pairwise frequencies are maintained using a key–item representation, where the key is a node pair and the item is its frequency. This representation is also implemented using two vectors, so that the storage format remains comparable at the implementation level.

In all methods, node-pair indices are represented by `uint64_t`, node indices are represented by `uint32_t`, and co-activation similarity values are represented by `double`. Therefore, the compared methods use consistent low-level data types and comparable storage representations in the implementation.

This section provides the complete experimental results omitted from the main text due to space limitations. The results are organized following the structure of Section VII-B, so that each subsection here complements the corresponding discussion in the main text.

E. Additional Results for Efficiency

Table IV reports the detailed runtime breakdown of the proposed algorithms, complementing the efficiency discussion in Section VII-B2. For both *DCGS-P* and *DCGS-MC*, the total runtime is decomposed into three components, i.e., pilot sampling, certified sampling, and optimization.

For *DCGS-P*, certified sampling is the dominant runtime component across all reported datasets, while the optimization cost of weighted peeling remains relatively small. By contrast, for *DCGS-MC*, the optimization component is substantially more expensive and becomes the dominant runtime component on most datasets due to the repeated min-cut computations at the checkpoints. The only exception is *Hamsterster*, where certified sampling and optimization have comparable costs. This contrast explains why *DCGS-P* is generally faster than *DCGS-MC*, even though both methods follow the same two-stage sampling framework.

These results are consistent with the discussion in Section VII-B2: the main additional cost of *DCGS-MC* comes from solving the weighted densest-subgraph subproblems exactly, whereas *DCGS-P* uses the substantially cheaper weighted-peeling routine.

F. Additional Results for Approximation Ratio

This subsection presents the complete version of the approximation-ratio results corresponding to Section VII-B3. In particular, Figure 8 and Table V extend Figure 3 and Table II to the remaining datasets.

Overall, the pattern observed in Section VII-B3 persists across the remaining datasets. At the early checkpoints, the upper bound is still very loose and the lower bound often remains zero, so the approximation ratio is either $+\infty$ or extremely large. As the number of samples increases, the lower bound becomes positive and the upper bound tightens quickly. Consequently, for most datasets, the approximation ratio becomes meaningful from around the seventh or eighth checkpoint and then rapidly stabilizes over the subsequent checkpoints. For *DCGS-MC*, the ratio is generally smaller at the same checkpoint, which is consistent with its tighter theoretical guarantee. For *Orkut* and *LiveJournal*, the corresponding entries of *DCGS-MC* are unavailable because this method is already infeasible on these datasets under our memory limit.

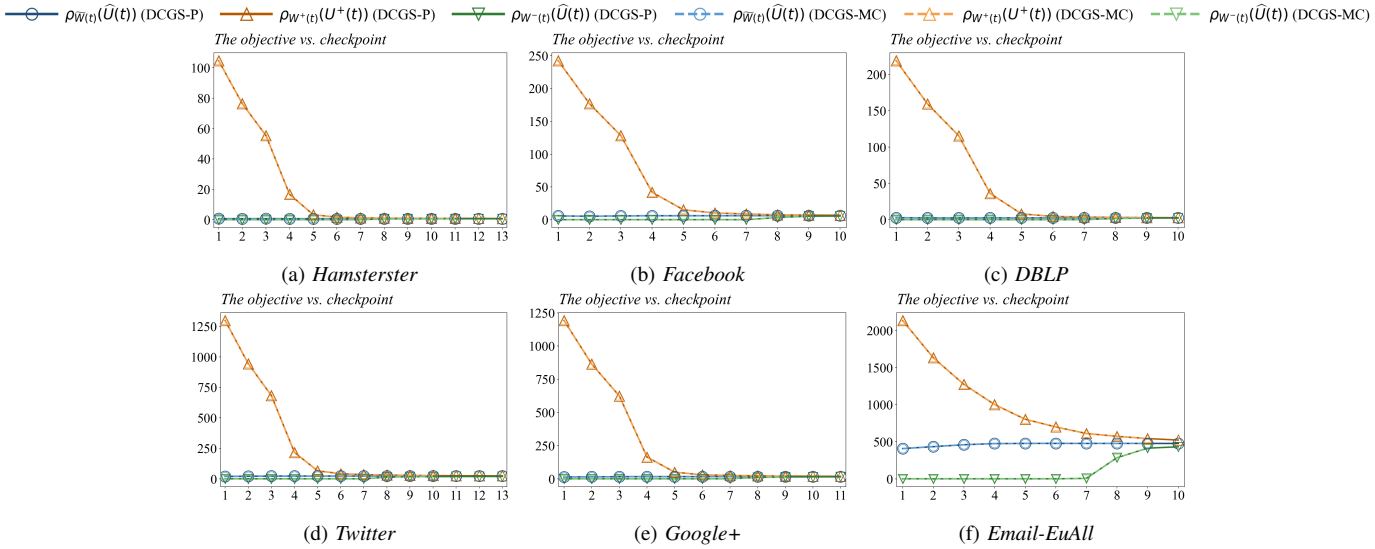
G. Additional Results for Parameter Sensitivity

This subsection presents the complete results for the parameter-sensitivity experiments corresponding to Section VII-B4 in the main text. For each parameter, we report

TABLE IV: Runtime breakdown of the proposed algorithms under the default parameter setting.

Dataset	*DCGS-P* Runtime (s)			*DCGS-MC* Runtime (s)		
	Pilot Sampling	Certified Sampling	Optimization	Pilot Sampling	Certified Sampling	Optimization
Hamsterster	0.332	75.2	0.0863	0.331	13.5	12.4
Facebook	2.04	41.8	0.563	2.04	20.7	88.5
DBLP	3.42	82.2	1.28	3.42	39.9	143
Douban	7.56	186	4.02	7.77	91.7	468
Twitter	48.2	20,376	68.5	48.4	1,810	3,212
Google+	51.7	4,175	32.9	51.8	1,050	4,328
Email-EuAll	64.4	2,587	41.4	64.2	1,312	3,472
Epinions	56.3	2,594	39.9	56.3	1,307	5,747
Pokec	131	9,905	191	131	4,968	16,924
Orkut	444	33,061	503	—	—	—
LiveJournal	804	55,774	1,006	—	—	—

Note. The reported runtime is decomposed into three parts, i.e., pilot sampling, certified sampling, and optimization. The sum of the three components equals the total runtime reported in Table I, up to minor measurement error.



(a) *Hamsterster* (b) *Facebook* (c) *DBLP*

(d) *Twitter* (e) *Google+* (f) *Email-EuAll*

Fig. 8: Changes in the estimated objective value and its upper and lower bounds across checkpoints.

the omitted figures and briefly summarize the corresponding trends.

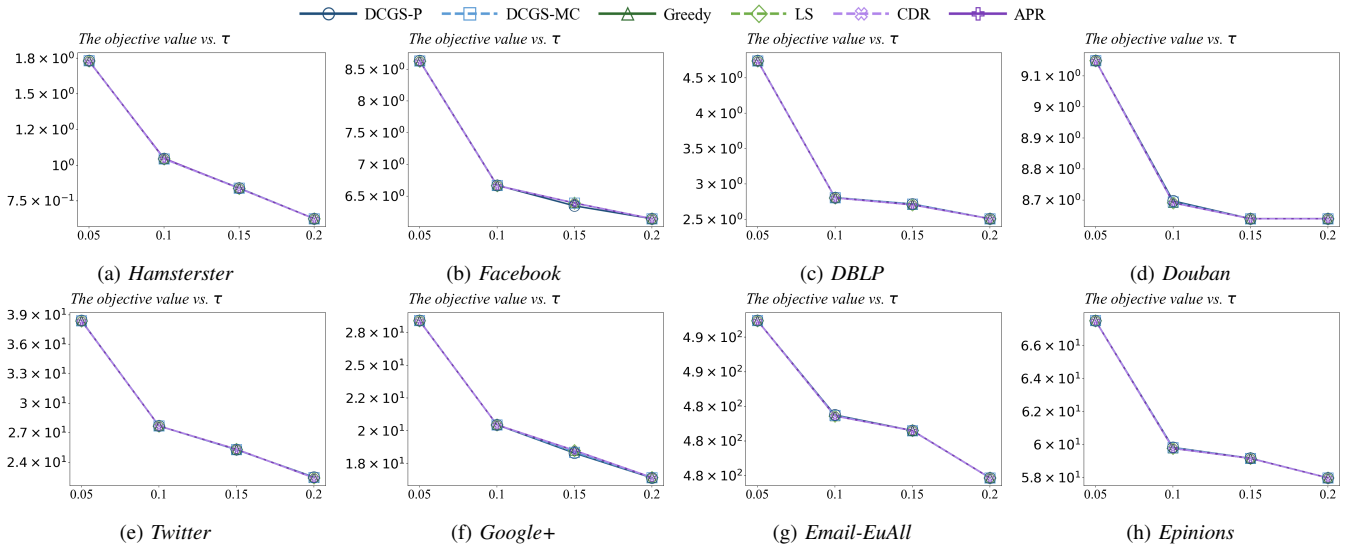
1) Additional Results for Varying τ: Figure 9 reports the objective values obtained with different co-activation thresholds τ. Across all datasets, the objective value exhibits a consistent downward trend as τ increases, since a larger threshold removes more node pairs with relatively low co-activation similarity. The objective value changes only slightly as τ decreases from 0.2 to 0.15 and then to 0.1. Although a larger relative improvement is observed when τ is further reduced from 0.1 to 0.05, the improvement remains small in absolute terms. Thus, within the evaluated range, using a positive threshold incurs only a modest loss in objective value relative to using a smaller threshold. The objective values of the compared methods also remain close across all threshold settings. This consistent performance further demonstrates the effectiveness of our algorithms under different choices of τ.

Figure 10 and Figure 11 present the complete results for varying τ, corresponding to the peak memory usage and runtime, respectively. Overall, these results are consistent with the discussion in the main text. In particular, decreasing τ generally leads to higher peak memory usage for both *DCGS-P* and *DCGS-MC*, with a more pronounced increase for *DCGS-MC*. By contrast, the effect of τ on runtime is less regular. For some datasets, including *Hamsterster*, *Facebook*, *DBLP*, and *Twitter*, the runtime does not decrease monotonically as τ increases. Nevertheless, the runtime variations remain within roughly the same order of magnitude, so the impact of τ is still much more pronounced on memory usage than on runtime.

2) Additional Results for Varying ε: Figure 12 and Figure 13 present the complete results for varying ε, corresponding to peak memory usage and runtime, respectively. These results are consistent with the discussion in the main text. In particular, increasing ε leads to a clear reduction in the runtime of *DCGS-P*, whereas its effect on *DCGS-MC* remains much less significant. This is consistent with the runtime breakdown in Section VII-B2: *DCGS-P* is mainly dominated by certified-stage sampling, while *DCGS-MC* incurs a much higher optimization cost. By contrast, the peak memory usage is only mildly affected by ε for both methods.

TABLE V: Approximation ratios at different checkpoints for the remaining datasets.

Dataset	Method	Checkpoint									
		t_1–t_4	t_5	t_6	t_7	t_8	t_9	t_{10}	t_{11}	t_{12}	t_{13}
Hamsterster	*DCGS-P*	$+\infty$	$+\infty$	$+\infty$	$+\infty$	3.82	3.40	3.15	2.97	2.84	2.41
	DCGS-MC	$+\infty$	$+\infty$	$+\infty$	$+\infty$	1.91	1.70	1.57	1.49	—	—
Facebook	*DCGS-P*	$+\infty$	$+\infty$	$+\infty$	861	4.56	2.62	2.42	—	—	—
	DCGS-MC	$+\infty$	$+\infty$	$+\infty$	430	2.28	1.31	—	—	—	—
DBLP	*DCGS-P*	$+\infty$	$+\infty$	$+\infty$	$+\infty$	3.82	2.61	2.42	—	—	—
	DCGS-MC	$+\infty$	$+\infty$	$+\infty$	$+\infty$	1.91	1.31	—	—	—	—
Twitter	*DCGS-P*	$+\infty$	$+\infty$	$+\infty$	244	4.67	3.23	2.93	2.75	2.64	2.47
	DCGS-MC	$+\infty$	$+\infty$	$+\infty$	122	2.33	1.62	1.47	—	—	—
Google+	*DCGS-P*	$+\infty$	$+\infty$	$+\infty$	204	4.33	2.89	2.62	2.50	—	—
	DCGS-MC	$+\infty$	$+\infty$	$+\infty$	102	2.17	1.45	—	—	—	—
Email-EuAll	*DCGS-P*	$+\infty$	$+\infty$	343,450	213	4.02	2.63	2.42	—	—	—
	DCGS-MC	$+\infty$	$+\infty$	171,725	106	2.01	1.31	—	—	—	—
Pokec	*DCGS-P*	$+\infty$	$+\infty$	548,613	259	4.01	2.66	2.44	—	—	—
	DCGS-MC	$+\infty$	$+\infty$	274,307	129	2.01	1.33	—	—	—	—
Orkut	*DCGS-P*	$+\infty$	$+\infty$	$+\infty$	240	3.80	2.62	2.42	—	—	—
	DCGS-MC	—	—	—	—	—	—	—	—	—	—
LiveJournal	*DCGS-P*	$+\infty$	$+\infty$	611,712	315	4.25	2.66	2.42	—	—	—
	DCGS-MC	—	—	—	—	—	—	—	—	—	—



(a) *Hamsterster* (b) *Facebook* (c) *DBLP* (d) *Douban*

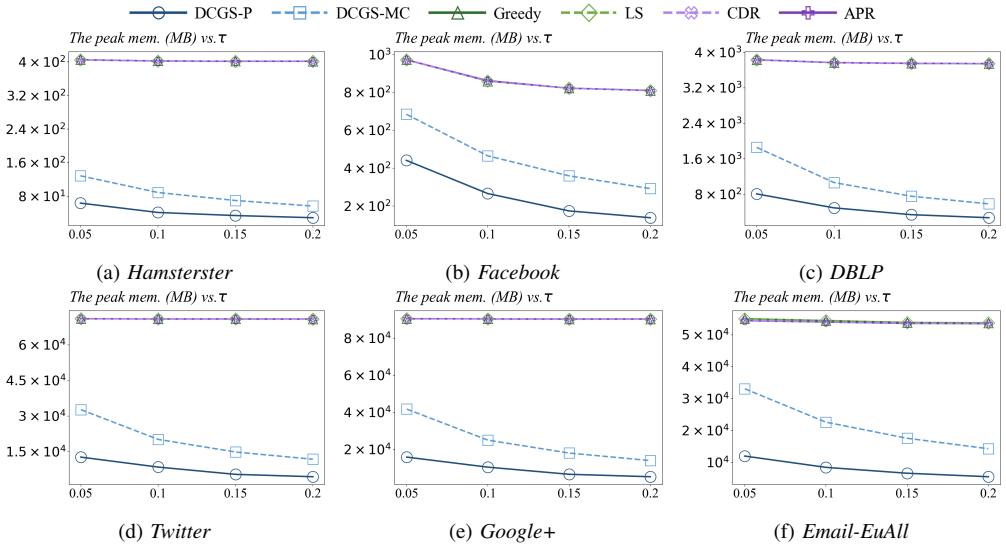
(e) *Twitter* (f) *Google+* (g) *Email-EuAll* (h) *Epinions*

Fig. 9: Objective values under different values of τ.

3) Additional Results for Varying δ: Figure 14 and Figure 15 present the complete results for varying δ, corresponding to peak memory usage and runtime, respectively. Overall, the effect of δ on peak memory usage is relatively stable: a larger δ generally leads to lower peak memory usage for both methods. By contrast, the effect of δ on runtime is less consistent across datasets. Overall, runtime tends to decrease as δ increases, although the trend is not monotonic on every dataset. Moreover, the impact of δ on peak memory usage is more pronounced for *DCGS-MC*, whereas its effect on runtime is more evident for *DCGS-P*. These observations are consistent with the discussion in the main text.

4) Additional Results for Varying S: Figure 16 and Figure 17 present the complete results for varying the size of the seed set S, corresponding to peak memory usage and runtime, respectively. The overall trend is clear: a larger seed set leads to substantially higher peak memory usage. In particular, when $|S| = 50$, the heuristic baselines already run out of memory on several medium-scale datasets, while *DCGS-MC* also incurs a considerable memory overhead, as shown in Figure 16. On the other hand, increasing $|S|$ also leads to runtime increases by orders of magnitude, as shown in Figure 17. These results suggest that designing algorithms for DCGS that remain efficient, memory-efficient, and theoretically grounded under settings with large seed sets is an important direction for future work.

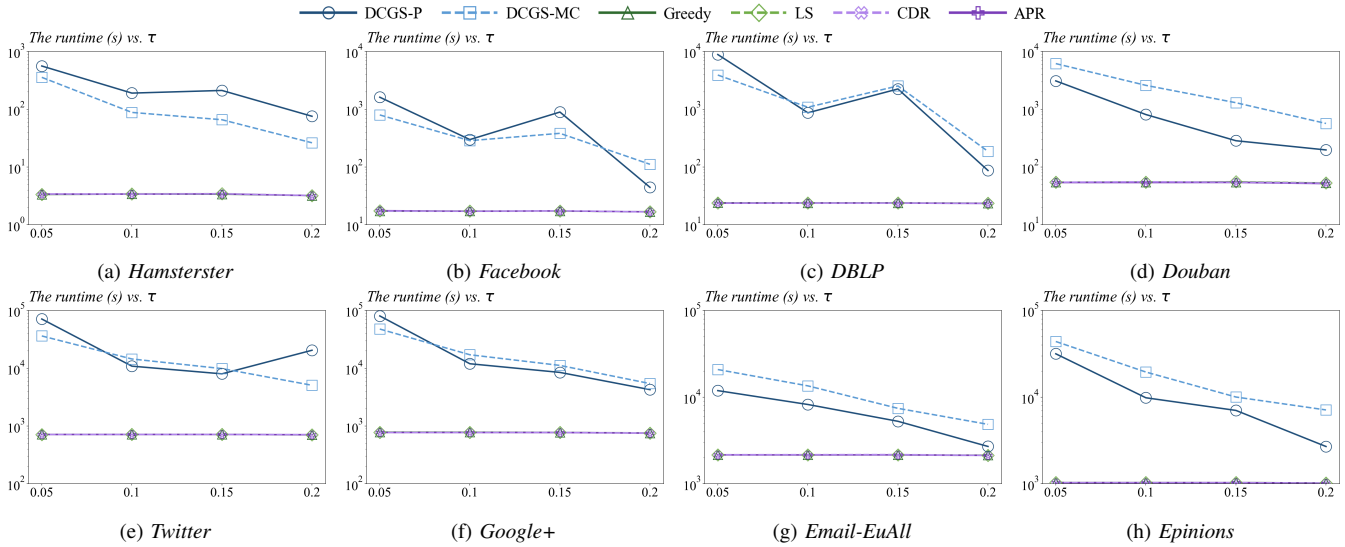
5) Additional Results for Varying $\mathcal{D}$: We further report the complete results under the IC model in Table VI and Table VII. Overall, these results are consistent with the discussion in the main text. In particular, all compared methods still achieve the same objective value on each dataset, while *DCGS-P* remains substantially more memory-efficient than the



(a) *Hamsterster* (b) *Facebook* (c) *DBLP*

(d) *Twitter* (e) *Google+* (f) *Email-EuAll*

Fig. 10: Peak memory usage under different values of τ.



(a) *Hamsterster* (b) *Facebook* (c) *DBLP* (d) *Douban*

(e) *Twitter* (f) *Google+* (g) *Email-EuAll* (h) *Epinions*

Fig. 11: Runtime under different values of τ.

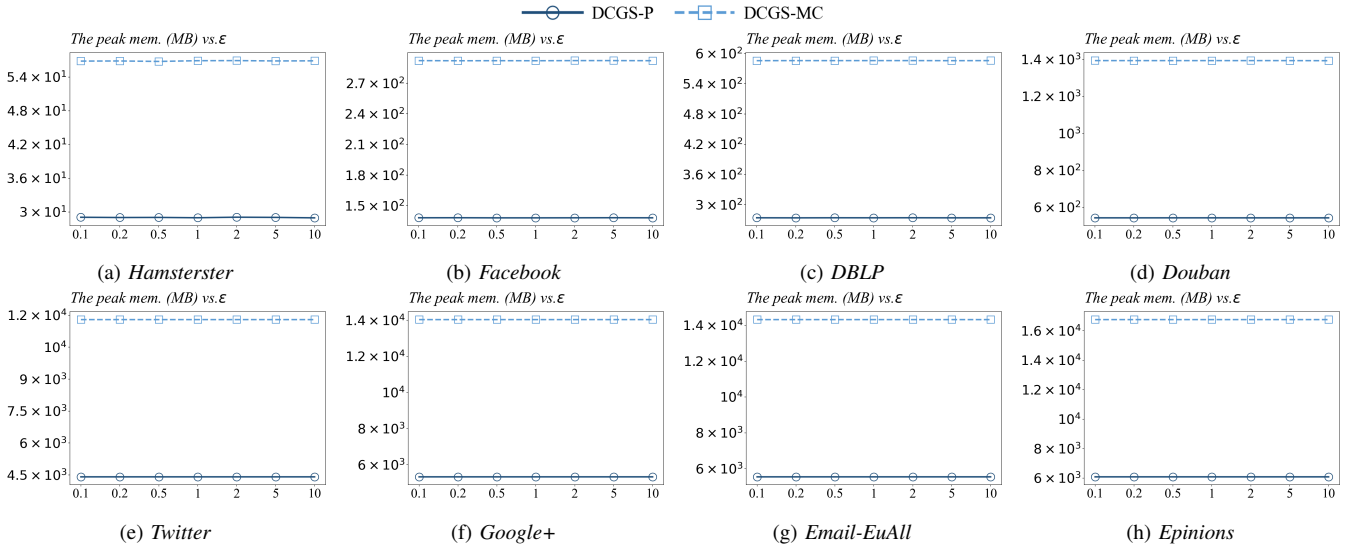
heuristic baselines and generally more practical than *DCGS-MC*. Moreover, Table VII shows that the runtime pattern also remains similar: for both proposed methods, the certified stage is still the dominant part of the total runtime, and the optimization overhead of *DCGS-MC* is again much higher than that of *DCGS-P*. These results further confirm that the relative behavior of the compared methods is stable under different diffusion models.

H. Additional Results under the Same Approximation Target

Table VIII reports the comparison between *DCGS-P* and *DCGS-MC* under the same target approximation ratio, i.e., 2.5. To this end, we keep the default setting for *DCGS-P* and set $\varepsilon = 1.5$ for *DCGS-MC*, so that both methods are evaluated at the stopping point for the 2.5-approximation criterion. Overall, the results are consistent with the discussion in the main text: even under the same approximation target, *DCGS-MC* still incurs higher peak memory usage than *DCGS-P* on all reported datasets, and its total runtime is also generally higher. Moreover, the runtime breakdown shows a similar pattern to that in Section VII-B2, i.e., *DCGS-P* is still mainly dominated by certified sampling, whereas *DCGS-MC* continues to incur a much larger optimization cost. These results further confirm that the practical advantage of *DCGS-P* does not come merely from using a looser target, but remains evident even when the two methods are compared under the same approximation criterion.

I. Additional Results for the Ablation Study

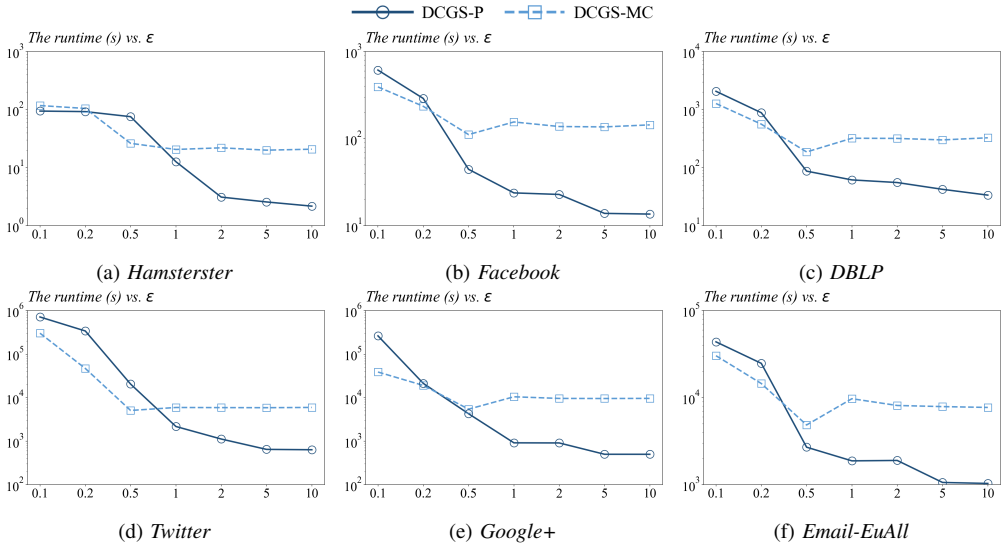
This subsection complements the ablation study in Section VII-B7 by reporting the complete results on eight datasets that are feasible for both the proposed method and the uncer-



(a) *Hamsterster* (b) *Facebook* (c) *DBLP* (d) *Douban*

(e) *Twitter* (f) *Google+* (g) *Email-EuAll* (h) *Epinions*

Fig. 12: Peak memory usage under different values of ε.

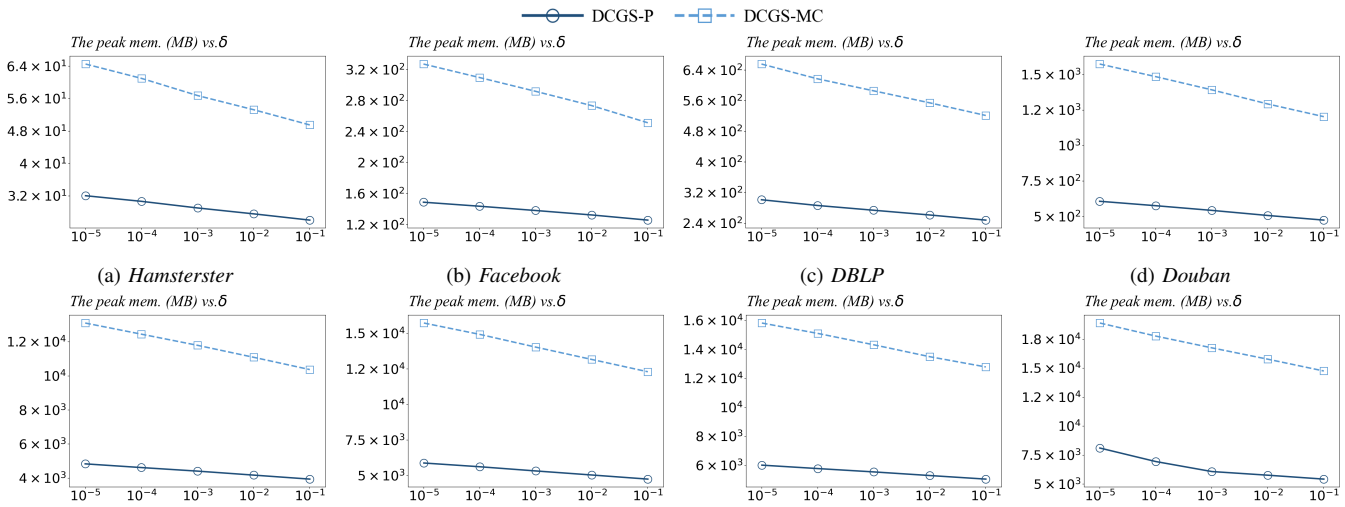


(a) *Hamsterster* (b) *Facebook* (c) *DBLP*

(d) *Twitter* (e) *Google+* (f) *Email-EuAll*

Fig. 13: Runtime under different values of ε.

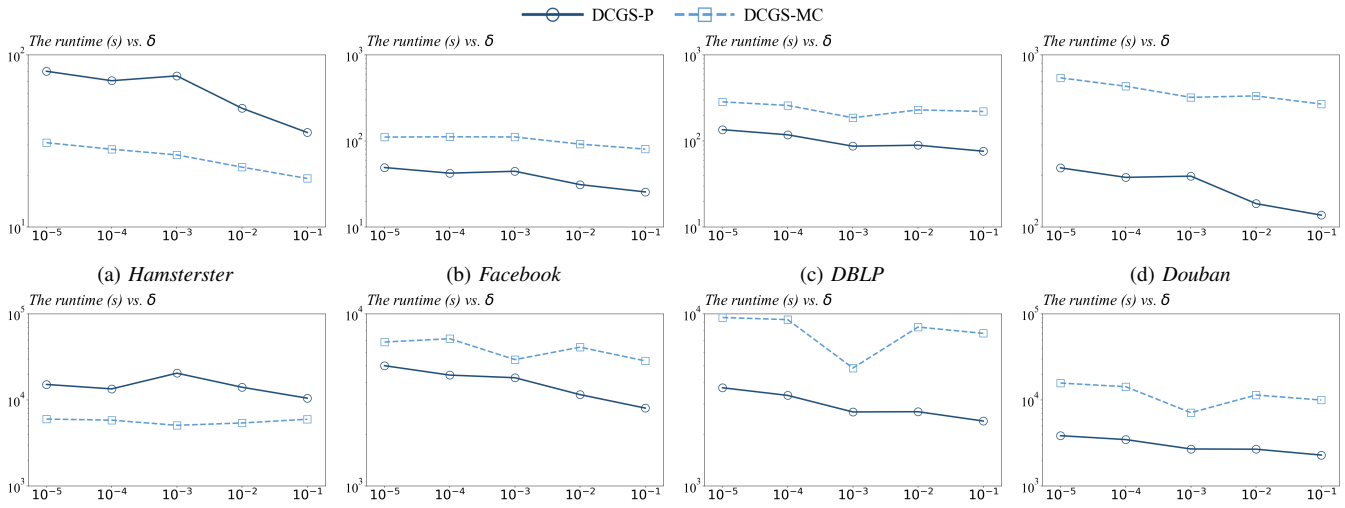
tified variant. Recall that the ablation variant keeps the pilot stage unchanged, but replaces the certified stage with a fixed budget of 10,000 diffusion realizations followed by weighted peeling. Table IX reports its objective value, runtime, and peak memory usage, together with the signed percentage changes in runtime and peak memory usage relative to *DCGS-P* and *Greedy*. In these percentage columns, a positive value means that the ablation variant is faster or more memory-efficient than the reference method, whereas a negative value means that it is slower or uses more memory. Consistent with the observation in Section VII-B7, the uncertified variant attains the same objective values as the proposed method on all eight datasets. At the same time, it is substantially faster than *DCGS-P* on all datasets, while its peak memory usage remains far below that of *Greedy*. These results further suggest that the pilot stage is the main source of memory reduction, whereas the certification mechanism mainly contributes explicit guarantees.



(a) *Hamsterster* (b) *Facebook* (c) *DBLP* (d) *Douban*

(e) *Twitter* (f) *Google+* (g) *Email-EuAll* (h) *Epinions*

Fig. 14: Peak memory usage under different values of δ.



(a) *Hamsterster* (b) *Facebook* (c) *DBLP* (d) *Douban*

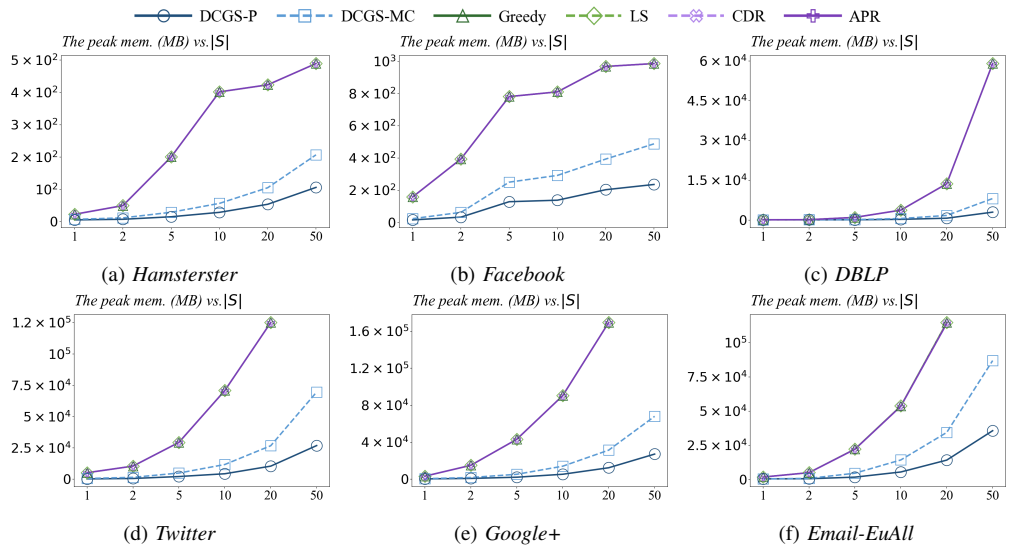
(e) *Twitter* (f) *Google+* (g) *Email-EuAll* (h) *Epinions*

Fig. 15: Runtime under different values of δ.

TABLE VI: Performance comparison under the IC model (sorted by peak memory usage).

Dataset	Obj. Value	Runtime (s)			Peak Mem. (MB)		
		DCGS-P	*DCGS-MC*	*Greedy*	*DCGS-P*	*DCGS-MC*	*Greedy*
Hamsterster	0.595	12.6	30.8	**3.28**	**25.8**	49.5	412
Facebook	6.12	47.3	148	**17.6**	**132**	269	771
DBLP	2.50	57.3	139	**22.5**	**250**	536	3,427
Douban	8.63	204	938	**55.0**	**517**	1,329	13,741
Twitter	20.9	5,247	4,055	**674**	**4,063**	10,718	62,154
Google+	14.4	6,329	7,328	**755**	**4,941**	12,902	84,526
Email-EuAll	475	2,457	5,945	**2,129**	**5,294**	13,661	52,689
Epinions	57.4	4,185	7,239	**986**	**5,562**	15,248	96,256
Pokec	64.7	**14,854**	19,954	—	**20,625**	59,480	OOM
Orkut	46.7	**26,128**	—	—	73,810	OOM	OOM
LiveJournal	65.8	**89,676**	—	—	122,914	OOM	OOM

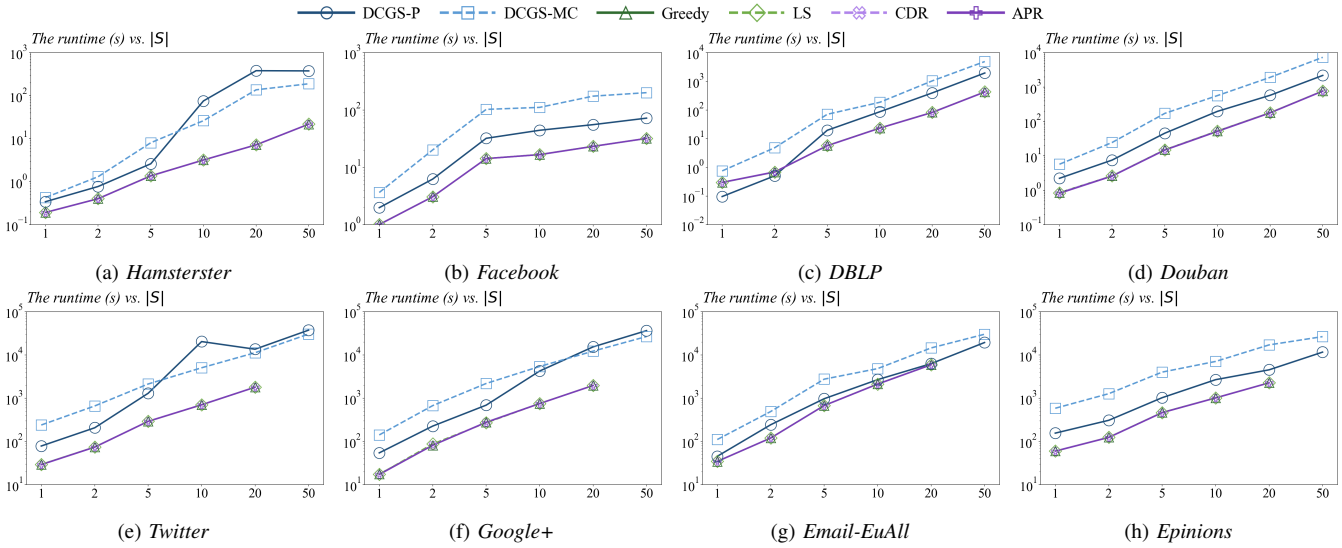
Note. Under this setting, all six compared algorithms achieve the same objective value on each dataset. Hence, the table reports only one objective value per dataset, taken from *DCGS-P*. In addition, the runtime and peak memory usage of *LS*, *CDR*, and *APR* are omitted for brevity, since they are identical to those of *Greedy* on the same dataset. OOM means that the memory usage exceeds the limit of 200 GB = 204,800 MB.



(a) *Hamsterster* (b) *Facebook* (c) *DBLP*

(d) *Twitter* (e) *Google+* (f) *Email-EuAll*

Fig. 16: Peak memory usage under different sizes of S.



(a) *Hamsterster* (b) *Facebook* (c) *DBLP* (d) *Douban*

(e) *Twitter* (f) *Google+* (g) *Email-EuAll* (h) *Epinions*

Fig. 17: Runtime under different sizes of S.

TABLE VII: Runtime breakdown of the proposed algorithms under the IC model.

Dataset	*DCGS-P* Runtime (s)			*DCGS-MC* Runtime (s)		
	Pilot Sampling	Certified Sampling	Optimization	Pilot Sampling	Certified Sampling	Optimization
Hamsterster	0.318	12.1	0.110	0.310	11.5	19.0
Facebook	2.11	44.4	0.827	2.11	20.0	126
DBLP	3.27	53.1	1.00	3.27	26.8	109
Douban	8.17	189	6.35	8.51	89.9	840
Twitter	45.4	5,175	27.0	45.4	1,295	2,714
Google+	49.0	6,249	31.9	48.8	3,146	4,134
Email-EuAll	64.8	2,346	45.5	64.8	1,146	4,735
Epinions	55.9	4,085	44.3	55.5	965	6,219
Pokec	127	14,556	171	128	3,699	16,127
Orkut	447	25,155	526	—	—	—
LiveJournal	795	87,862	1,019	—	—	—

Note. The reported runtime is decomposed into three parts, i.e., pilot sampling, certified sampling, and optimization. The sum of the three components equals the total runtime reported in Table VI, up to minor measurement error.

TABLE VIII: Performance comparison at the stopping point for the 2.5-approximation criterion across all datasets.

Dataset	Method	Obj. Value	Runtime (s)				Peak Mem. (MB)
			Pilot Sampling	Certified Sampling	Optimization	Total	
Hamsterster	*DCGS-P*	0.625	0.332	75.2	0.0863	75.6	27.7
	DCGS-MC	0.625	0.335	2.38	12.9	15.6	54.2
Facebook	*DCGS-P*	6.14	2.04	41.8	0.563	44.4	132
	DCGS-MC	6.14	2.09	8.63	93.1	104	279
DBLP	*DCGS-P*	2.51	3.42	82.2	1.28	87.0	261
	DCGS-MC	2.51	3.39	17.4	118	139	559
Douban	*DCGS-P*	8.64	7.76	186	4.02	197	518
	DCGS-MC	8.64	7.71	35.7	399	442	1,329
Twitter	*DCGS-P*	22.5	48.2	20,376	68.5	20,493	4,207
	DCGS-MC	22.4	48.4	543	2,914	3,505	11,250
Google+	*DCGS-P*	16.4	51.7	4,175	32.9	4,259	5,086
	DCGS-MC	16.4	51.3	473	4,105	4,629	13,388
Email-EuAll	*DCGS-P*	477	64.4	2,587	41.4	2,693	5,282
	DCGS-MC	477	63.5	535	3,058	3,656	13,661
Epinions	*DCGS-P*	58.0	56.3	2,594	39.9	2,690	5,802
	DCGS-MC	58.0	55.7	487	5,159	5,702	15,983
Pokec	*DCGS-P*	66.2	131	9,905	191	10,226	21,030
	DCGS-MC	66.2	130	1,900	15,860	17,890	61,058

TABLE IX: Complete results of the ablation study. The columns "Runtime red." and "Mem. red." report the signed percentage change relative to the reference method. A positive value indicates that the ablation variant reduces runtime or peak memory usage, whereas a negative value indicates that it increases runtime or peak memory usage.

Dataset	Obj. Value	Runtime (s)	Peak Mem. (MB)	vs. *DCGS-P*		vs. *Greedy*	
				Runtime red.	Mem. red.	Runtime red.	Mem. red.
Hamsterster	0.625	3.13	26.8	+95.9%	+3.3%	+2.1%	+93.0%
Facebook	6.14	20.3	140	+54.3%	-6.2%	-21.5%	+81.9%
DBLP	2.51	43.8	275	+49.6%	-5.3%	-86.3%	+92.3%
Douban	8.64	93.4	584	+52.7%	-12.9%	-77.2%	+95.8%
Twitter	22.4	702	4,994	+96.6%	-18.7%	+0.3%	+92.6%
Google+	16.4	720	5,952	+83.1%	-17.0%	+4.6%	+93.1%
Email-EuAll	477	1,128	6,155	+58.1%	-16.5%	+47.1%	+88.0%
Epinions	58.0	904	7,107	+66.4%	-22.5%	+11.3%	+92.7%